Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Embedded Systems Engineering
State Charts

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

Überblick über die gesamte Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Eingebettete Systeme

- Definition
- Eigenschaften
- Anforderungen

Entwurf Eingebetteter Systeme

Modellierungssprachen

- Modell-basierte Entwicklung
- Endliche Automaten
- Statecharts
- Timed Automata
- Signalfluss-orientierte Sprachen (Simulink)
- SystemC

Implementierung

- Programmiersprachen
- Betriebssysteme
- Synthese

Validierung und Verifikation

- Testen
- Simulation
- Model Checking

Überblick über die gesamte Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Eingebettete Systeme

Entwurf Eingebetteter Systeme

Modellierungssprachen

- Endliche Automaten
- Statecharts

Implementierung

Validierung und Verifikation

Übersicht über aktuelle Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

1. Endliche Automaten
2. Statecharts
   - Statechart-Konstrukte
   - Einschub: Syntax versus Semantik
   - Statechart-Semantik
   - Beispiele

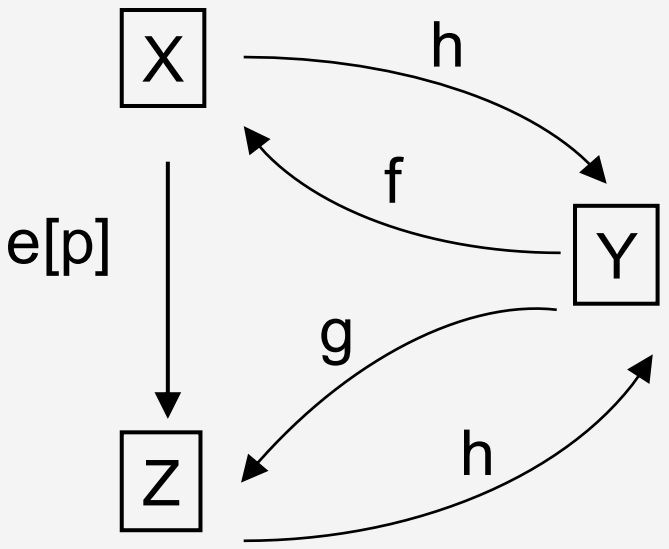
Übersicht über aktuelle Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

1. Endliche Automaten
2. Statecharts
   - Statechart-Konstrukte
   - Einschub: Syntax versus Semantik
   - Statechart-Semantik
   - Beispiele

Recap: Endliche Automaten

- *Finite State Machines* (FSM)
- Zustandsdiagramm
- Übergänge markiert mit
  - Externen Ereignissen (f,g,h,e)
  - Bedingungen (p)
- Kann Ereignisse aussenden
- Beispiel:
  - Ereignis e bewirkt Übergang von Zustand X nach Zustand Z, falls Bedingung p erfüllt ist.



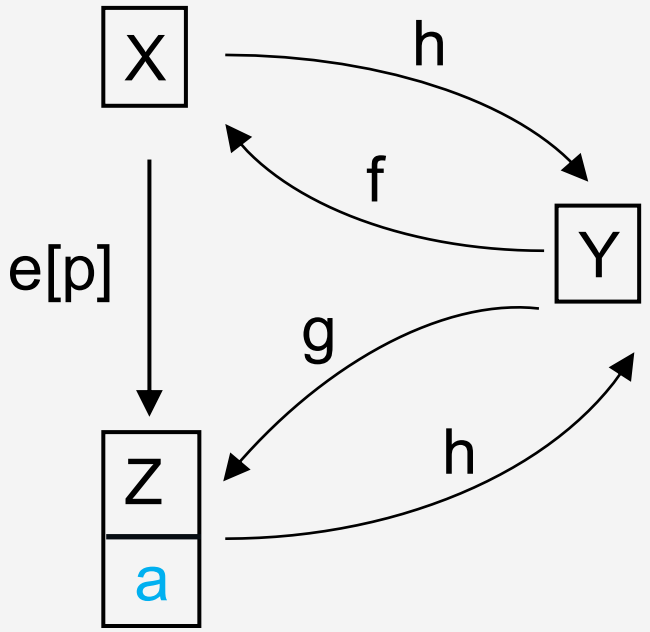
Endliche Automaten

Zwei Arten von FSMs: *Moore* und *Mealy*

- Für beide gilt:
  - Kuller/Kästchen = Zustände (*Vertices*)
  - Pfeile = Übergänge/Transitionen (*Edges*)
  - Transitionen werden durch Ereignisse ausgelöst und durch Bedingungen (*Guards*) abgesichert
- Unterschiede beim Aussenden von Ereignissen

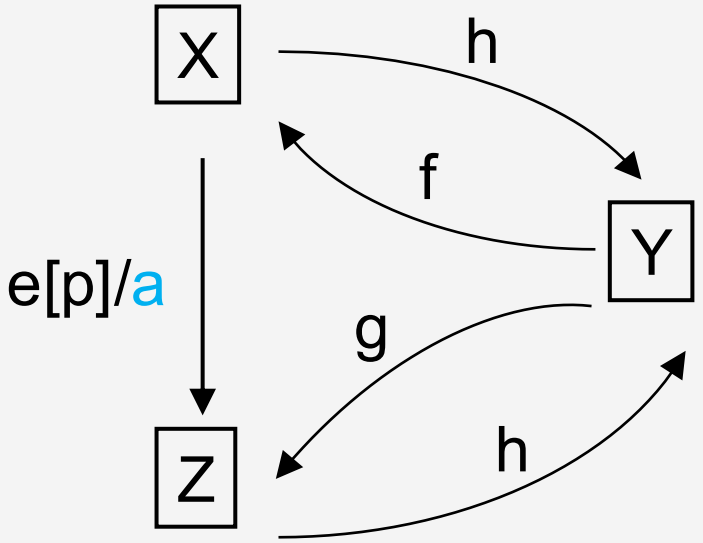
Moore Automaten

- Ausgabeereignisse in den Zuständen
- Wenn der Automat im Zustand X ist
  und Ereignis e auftritt
  und Bedingung p erfüllt ist,
  gelangt der Automat in den Zustand Z.
- **Solange** Automat in Zustand Z wird das Ereignis a gesendet (beliebig viele Takte)



Mealy Automaten

- Ausgabeereignisse an den Transitionen
- Wenn der Automat im Zustand X ist
  und Ereignis e auftritt
  und Bedingung p erfüllt ist,
  gelangt der Automat in den Zustand Z.
- Während der Transition wird **einmalig** Ereignis a gesendet (genau ein Takt).



Mathematisches Modell (Syntax)

$$EA = (S, s_0, \Sigma, \Gamma, \delta, \omega)$$

S endliche Menge an Zuständen

s_0 eindeutiger Startzustand (aus S)

Σ endliche Menge an Eingabeereignissen

Γ endliche Menge an Ausgabeereignissen

δ Zustandsübergangsfunktion ($S\ x\ \Sigma \rightarrow S$)

ω Ausgabefunktion ($S\ x\ \Sigma \rightarrow \Gamma$ oder $S \rightarrow \Gamma$)

↑ Mealy ↑ Moore

Beispiel: Drehsperre

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Bestandteile:

- **Münzeingabe:** akzeptiert nur 1€-Münzen
- **Bewegungssensor:** wird aktiviert, wenn sich die Sperre um 120° dreht

Verhalten:

- Wird eine **Münze eingeworfen**, soll die **Drehsperre entriegelt** werden.
- Während einer **Drehung** soll die **Drehsperre verriegelt** sein.

Quelle: Wikimedia Commons

Beispiel: Drehsperre

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Bestandteile:

- Münzeingabe: akzeptiert nur 1€-Münzen
- Bewegungssensor: wird aktiviert, wenn sich die Sperre um 120° dreht

Verhalten:

- Wird eine Münze eingeworfen, soll die Drehsperre entriegelt werden.
- Während einer Drehung soll die Drehsperre verriegelt sein.

Zustände

- Verriegelt, entriegelt
- **Initialer** Zustand: verriegelt

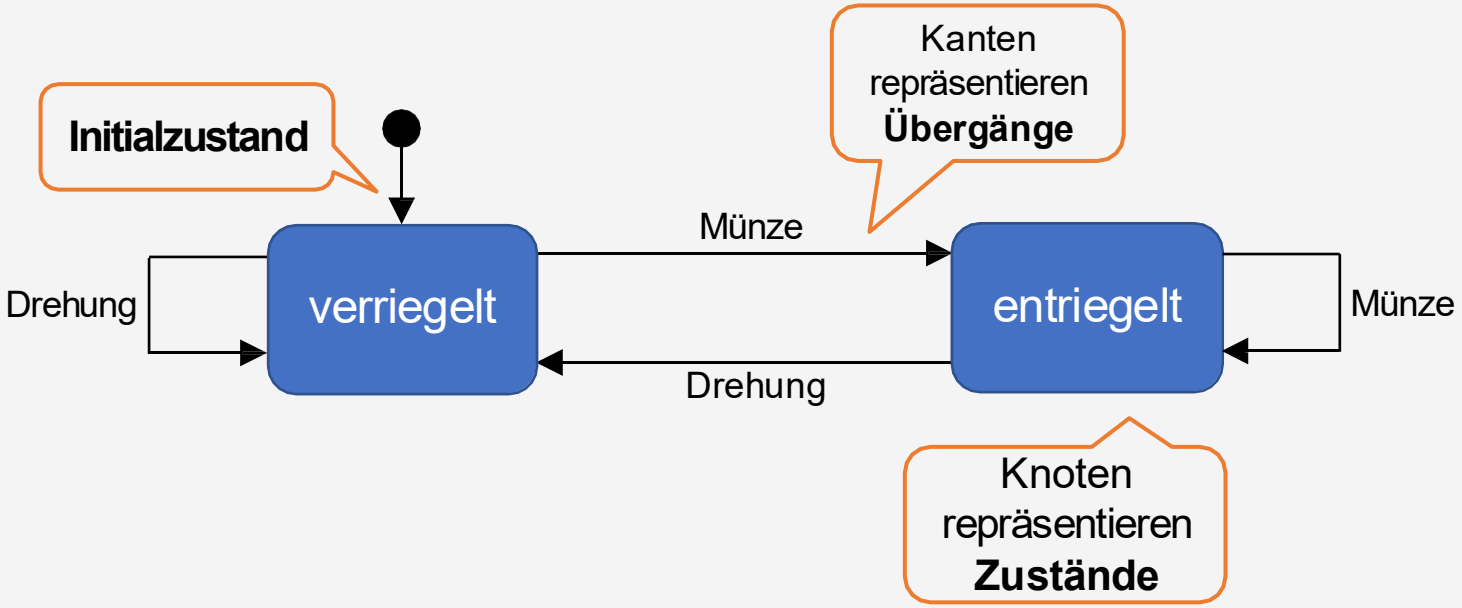
Ereignisse

- Münze, Drehung

Übergänge

	Münze	Drehung
verriegelt	entriegelt	verriegelt
entriegelt	entriegelt	verriegelt

Grafische Notation



Beispiel: Kaffeeautomat

- Ein Automat verkauft Kaffee für 2€, akzeptiert 1€- oder 2€-Münzen.
- Überflüssige Münzen werden sofort zurückgegeben.
- Sobald 2€ bezahlt wurden, verstaut der Automat die Münzen und fängt an Kaffee zu kochen (ein Becher wird automatisch ausgegeben).
- Nachdem der Kaffee fertig ist, wartet der Automat bis der Becher entnommen wird.
- Solange der Becher noch nicht entnommen wurde, gibt der Automat weitere eingeworfene Münzen zurück.

Welche Zustände und Ereignisse hat der Automat?

Zustände

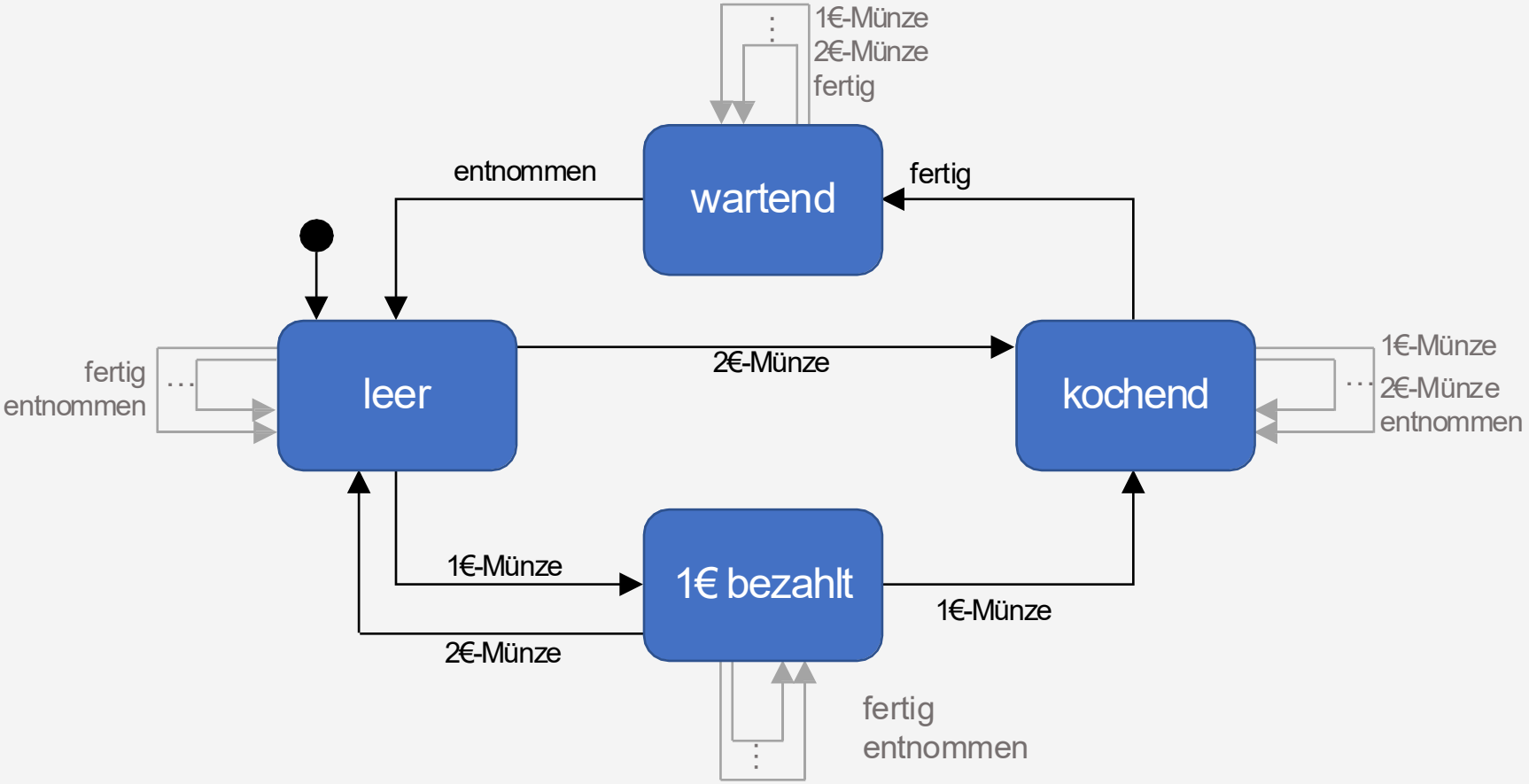
- **leer** / keine Münzen **(initial)**
- **1€ bezahlt**
- Kaffee **kochend**
- Mit Becher **wartend**

Ereignisse

- **1€-Münze** eingeworfen
- **2€-Münze** eingeworfen
- Kaffee **fertig** gekocht
- Becher **entnommen**

Und wie sieht der Automat aus?

Kaffeeautomat



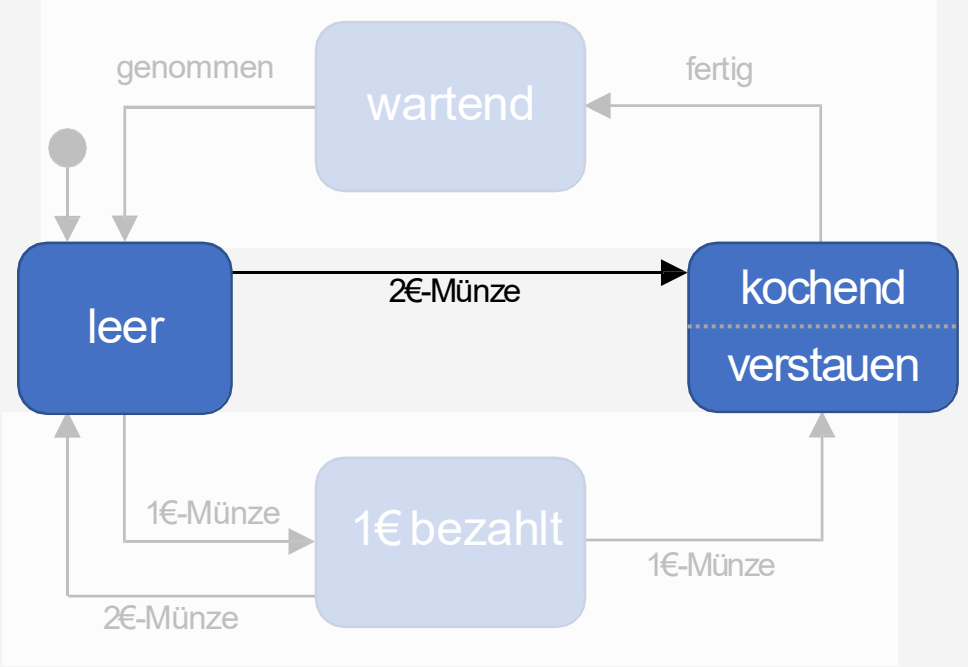
Bei reaktiven deterministischen Systemen kann man **„Self-Loops“ weglassen**:

Für alle Ereignisse, zu denen kein Übergang aus dem Zustand spezifiziert wird, wird ein Self-Loop angenommen.

Moore-Automaten

Wenn 2€ empfangen werden, muss **die Kontrolleinheit dem Münzschlitz mitteilen**, dass er das Geld **verstauen** soll.

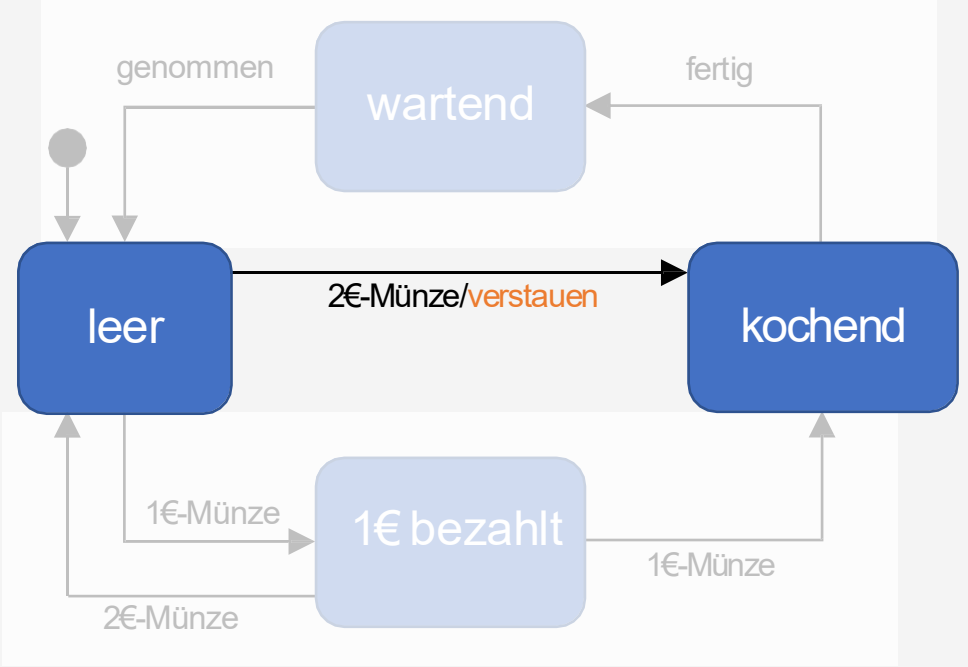
- Ausgabeereignisse **in den Zuständen**
- **Beim Betreten** des Zustands „kochend“ wird das Ereignis „verstauen“ gesendet



Mealy-Automaten

Wenn 2€ empfangen werden, muss **die Kontrolleinheit dem Münzschlitz mitteilen**, dass er das Geld **verstauen** soll.

- Ausgabeereignisse **an den Transitionen**
- **Während der Transition** wird das Ereignis „verstauen“ gesendet



Moore vs. Mealy

Welches Automatenmodell ist **mächtiger**?

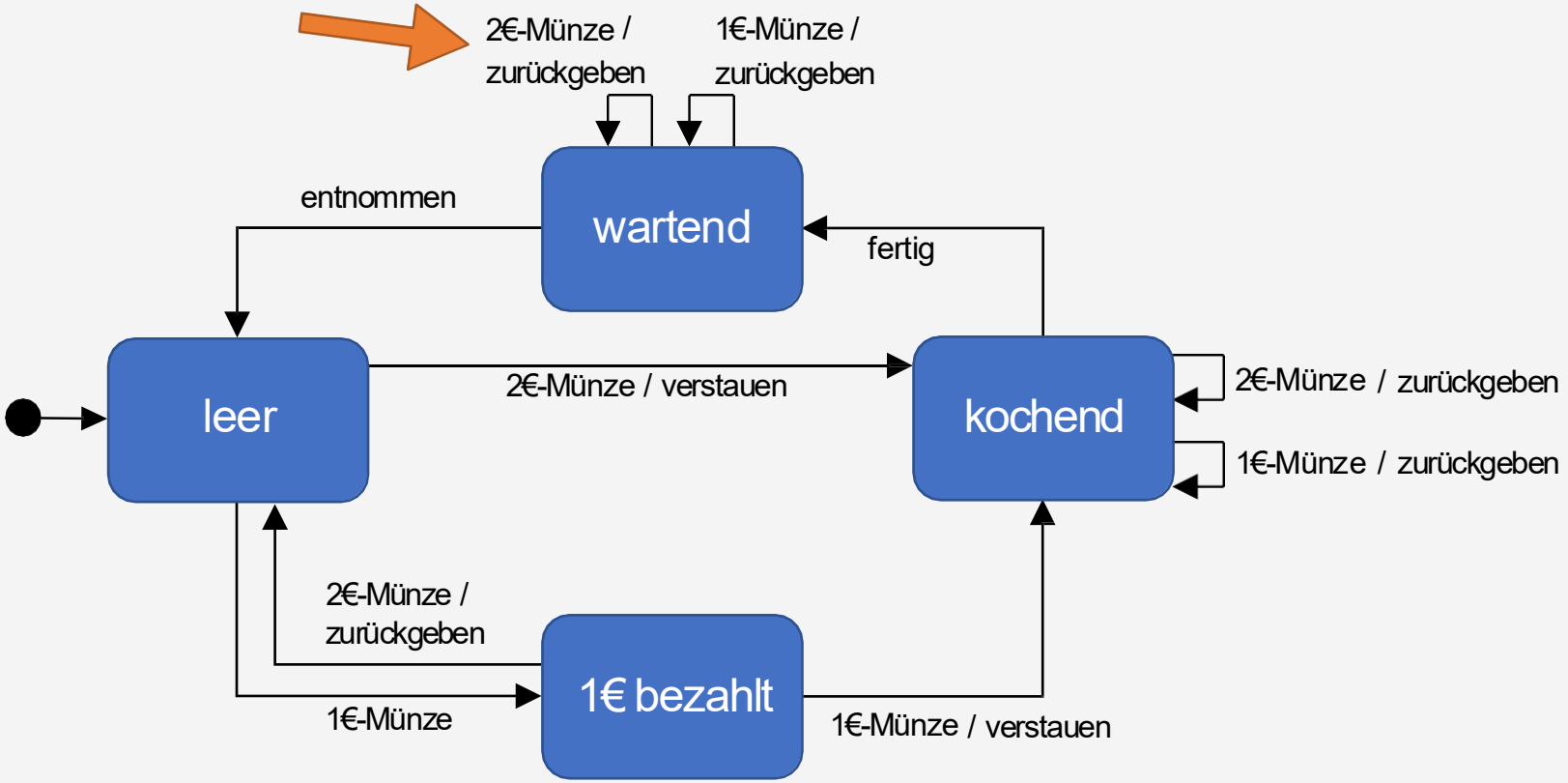
Keins. Beide Modelle lassen sich ineinander überführen, sind also gleich mächtig!

Welches Automatenmodell hat **mehr Zustände**?

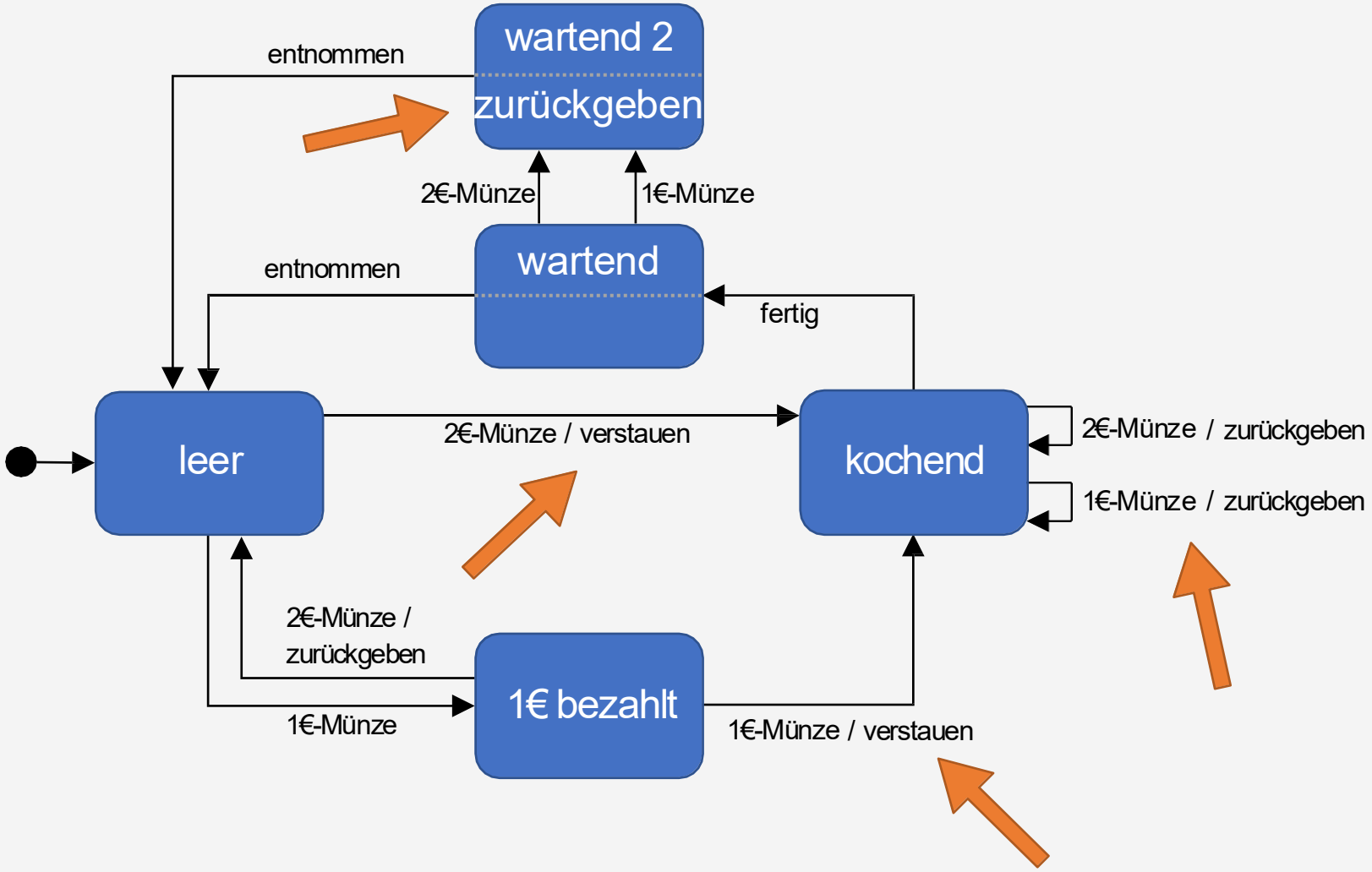
Moore Automaten besitzen <u>mindestens</u> so viele Zustände wie äquivalente Mealy Automaten!

Beispiel: Mealy nach Moore

Ein Mealy-Automat kann in einen äquivalenten Moore-Automaten überführt werden. Es entstehen aber häufig **zusätzliche Zustände**.

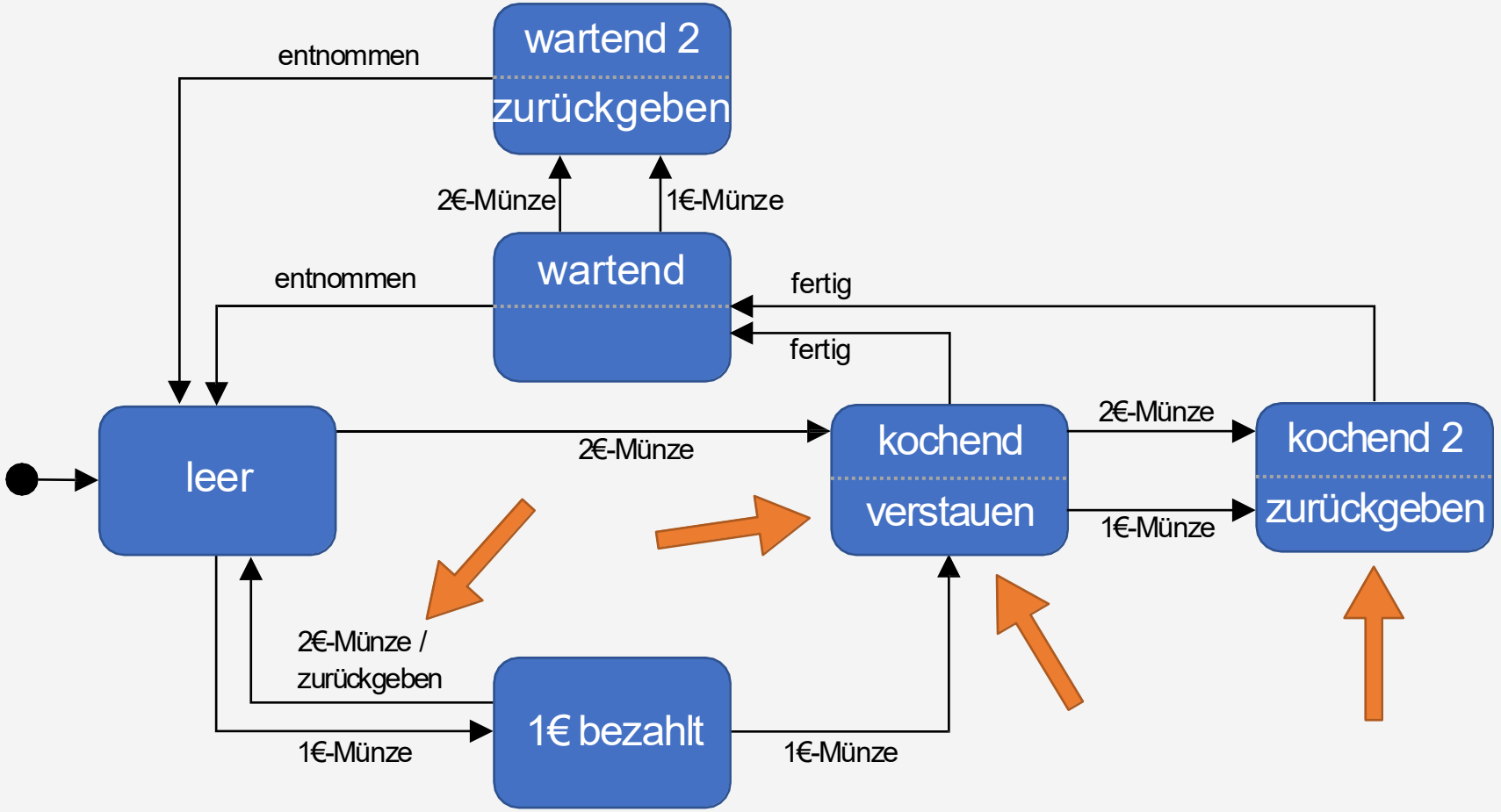


Beispiel: Mealy nach Moore



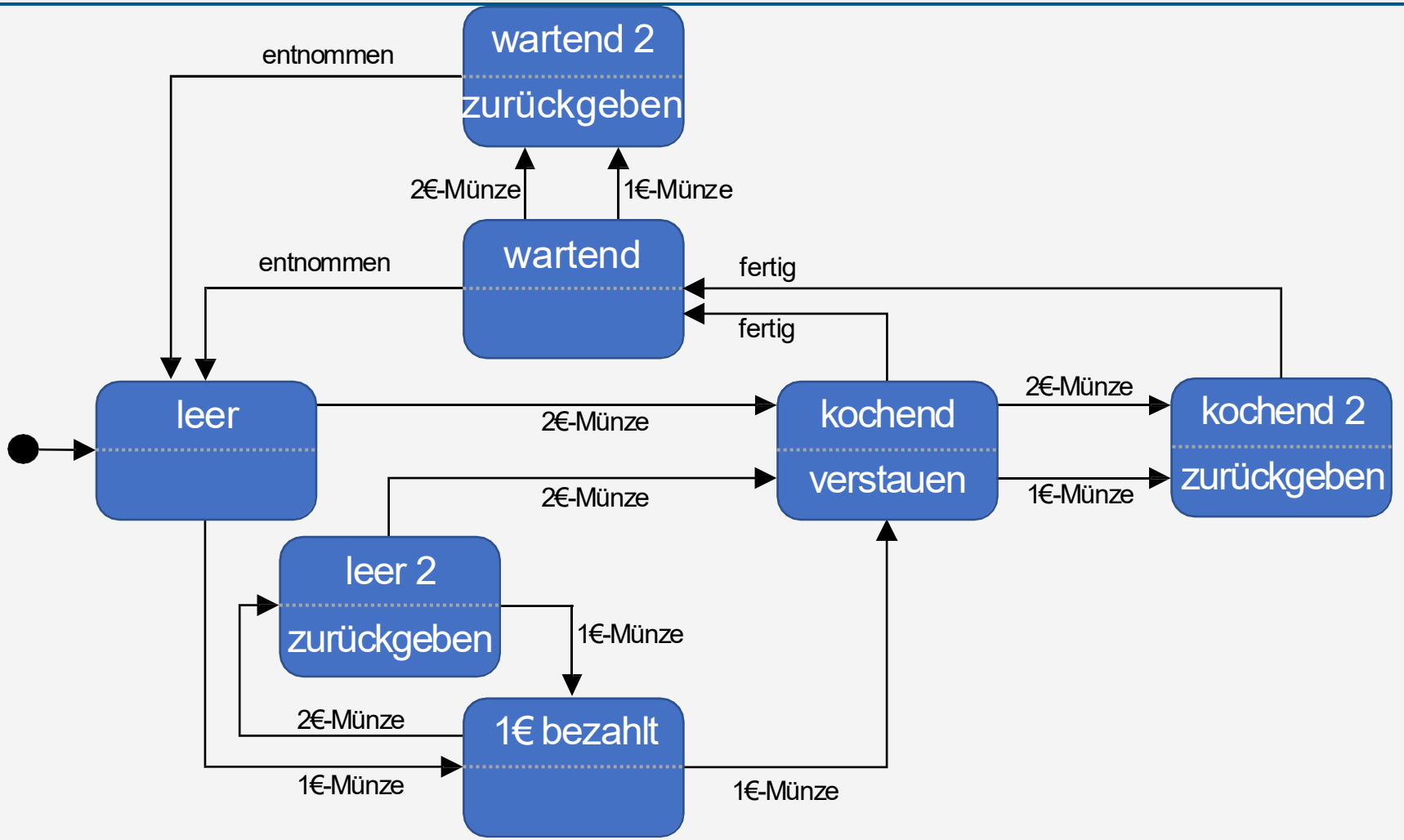
Mealy- nach Moore-Automat

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law



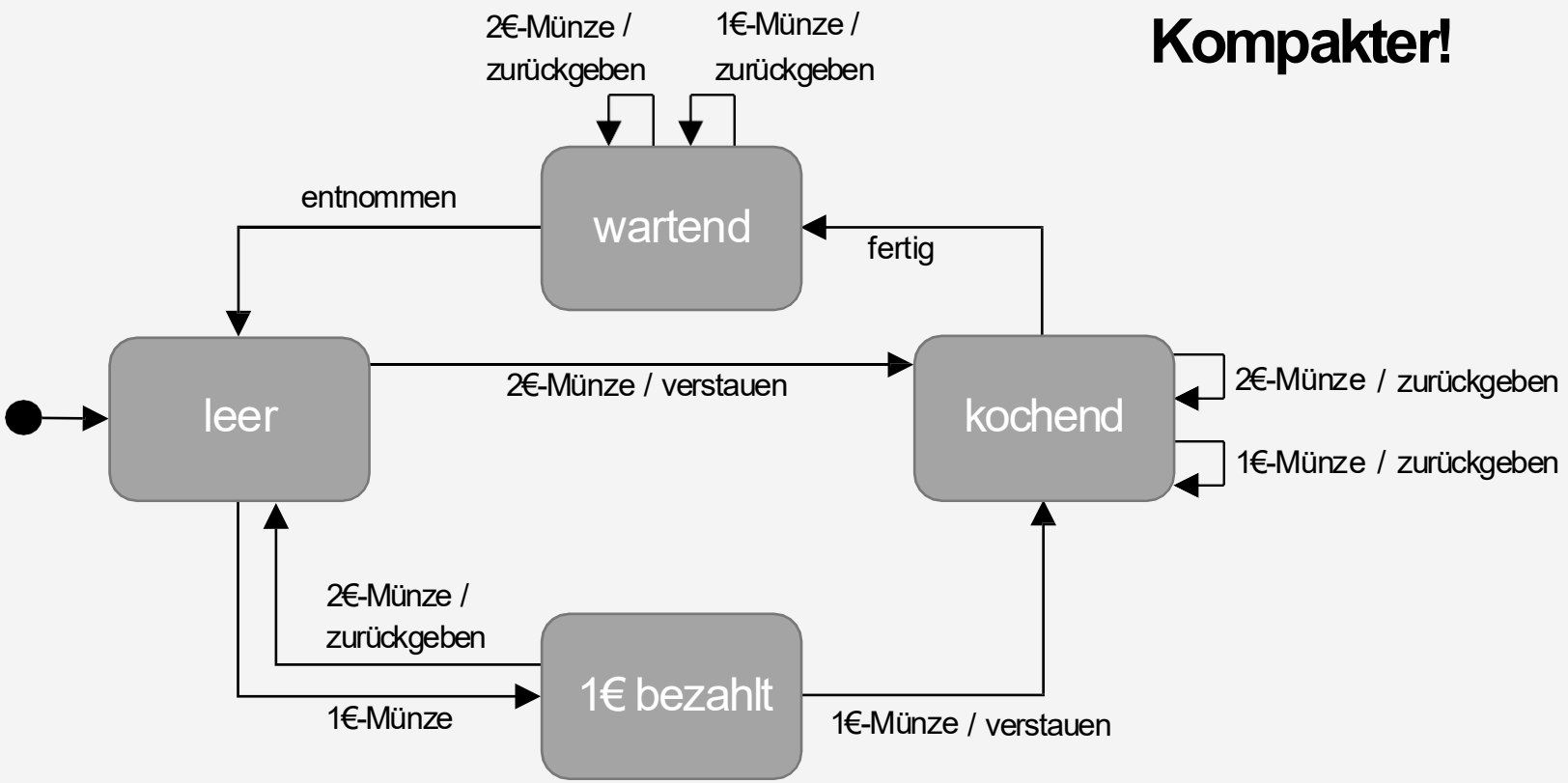
Mealy- nach Moore-Automat

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law



Vergleich: Mealy-Kaffeeautomat

Kompakter!



Automaten: Vor- und Nachteile

Diskutieren Sie Vor- und Nachteile von Endlichen Automaten für die Modellierung eingebetteter Systeme!

Automaten: Vor- und Nachteile

- ✓ Formale Semantik (Ermöglicht Verifikation)
- ✓ ausführbar
- ✓ Grafische Darstellung, sehr intuitiv
- ✓ Geeignet für Hardware und Software

- Nur atomare Zustände (hierarchischer Entwurf mit Verfeinerungen nicht möglich)
- **Zustandsexplosion** bei Nebenläufigkeit

Tool

https://www.jflap.org

Zustandsexplosion

Endliche Automaten haben keine Möglichkeit Nebenläufigkeit zu modellieren

- Ausmultiplizieren des gesamten möglichen Verhaltens nötig
- Exponentieller Anstieg der Anzahl an Zuständen

Zustandsexplosion

Beispiel

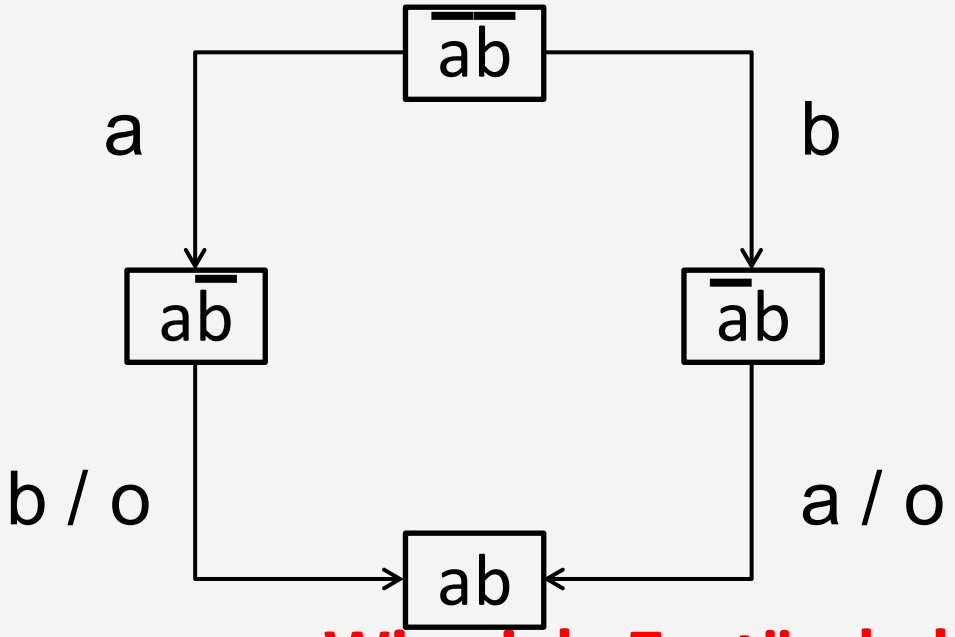
- Endlicher Automat wartet auf zwei Eingabesignale (a, b)
- Eingabesignale können in beliebiger Reihenfolge auftreten
- Automat sendet Ausgabesignal (o), wenn **beide** Eingabesignale angekommen sind

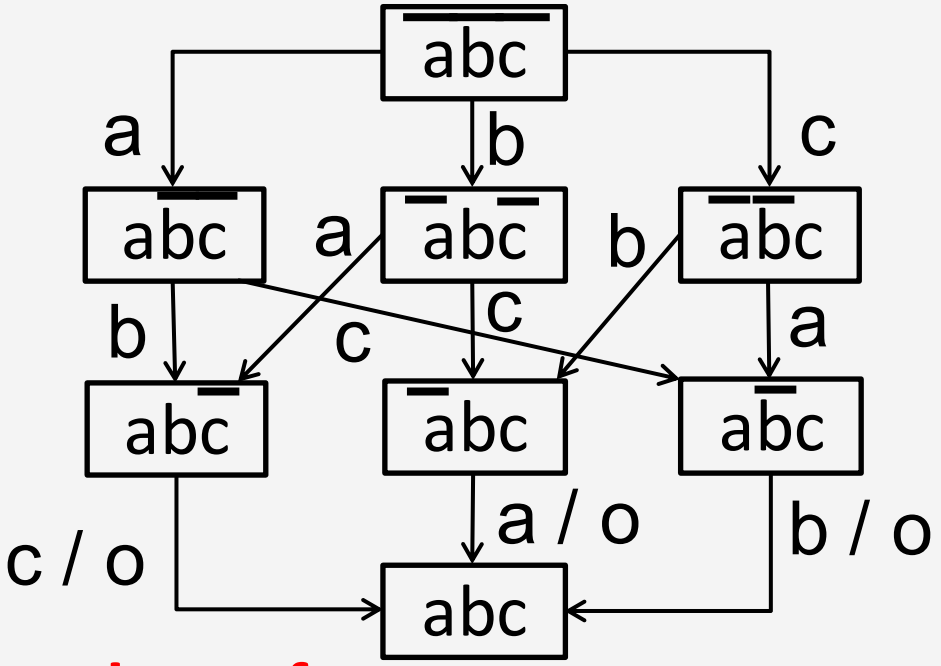
Wie viele Zustände hat der Automat?

Wie viele Zustände hat ein Automat, der auf 3 (4, 5, 100) Eingabesignale wartet?

Zustandsexplosion

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law





Wie viele Zustände hat ein Automat, der auf 3 (4, 5, 100) Eingabesignale wartet?

8 16 32 2^{100}

Übersicht über aktuelle Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

1. Endliche Automaten
2. Statecharts
   - Statechart-Konstrukte
   - Einschub: Syntax versus Semantik
   - Statechart-Semantik
   - Beispiele

Modellierung größerer Systeme

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Wie modelliert man folgende Zusammenhänge in Zustandsautomaten?

- In allen Fahrzuständen führt Bremsen zum Stillstand.
- Das Radio funktioniert gleich, egal ob man fährt oder steht
- Wenn die Kupplung betätigt wurde, wähle den Gang.
- Abfahren besteht aus Angurten, Zündung einschalten, Gang einlegen und Gas geben.

Modellierung größerer Systeme

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Wie modelliert man folgende Zusammenhänge in Zustandsautomaten?

- In allen Fahrzuständen führt Bremsen zum Stillstand.
- Das Radio funktioniert gleich, egal ob man fährt oder steht
- Wenn die Kupplung betätigt wurde, wähle den Gang.
- Abfahren besteht aus Angurten, Zündung einschalten, Gang einlegen und Gas geben.

Zusammenfassen von Zuständen zu einem Oberzustand

Erklären der Unabhängigkeit (Orthogonalität) von Zuständen

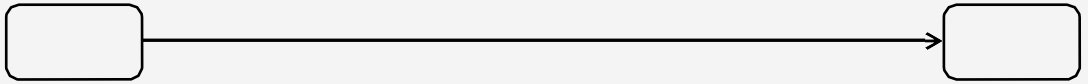
Verlangt nach Ausgabe, falls Vorbedingung erfüllt

Verfeinerung eines Zustands

Kantenbeschriftung

- Generelle Form der Kantenbeschriftung:

event [condition] / reaction



- *Event*: intern oder extern generiert
- *Condition*: Variablenbelegung (x = 5), interner Zustand (in X)
- *Reaction*: Variablenzuweisung (x := 7), Event senden

Statechart-Konstrukte

Erweiterungen in Zustandsdiagrammen:

- Oder-Zustand (XOR-Komposition)
- Und-Zustand (AND-Komposition)
- Hierarchie und Verfeinerung
- Broadcast
- Zusammenführung
- Historie
- Konditionalkonnektor
- Timer
- Entry-, Exit-, Throughout Actions

Oder-Zustand

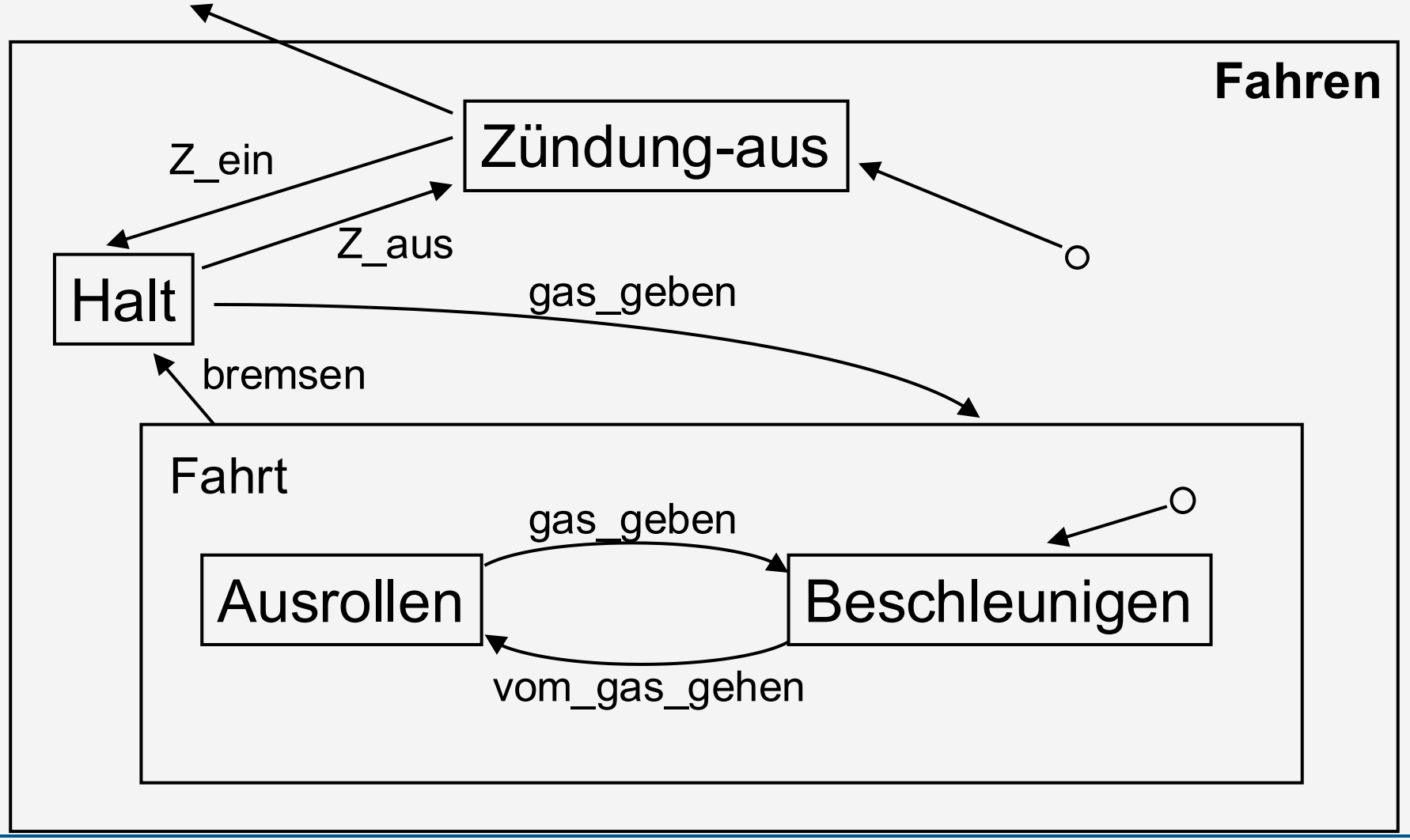
- Zusammenfassung eines Teils des Automaten zu einem Oberzustand
- Innerhalb Oberzustand:
  - interner Anfangszustand (Default-Zustand)
  - wird bei Übergang in Oberzustand eingenommen
  - auch Übergänge in bzw. aus beliebigen internen Zuständen heraus möglich
  - Oberzustand selbst ist kein Zustand, d.h. Automat ist immer in einem der internen Zustände
- Beispiel Auto: Alle Zustände, in denen PKW fährt, bilden Oberzustand. Ereignis „Bremsen“ führt heraus.

Hierarchie und Verfeinerung

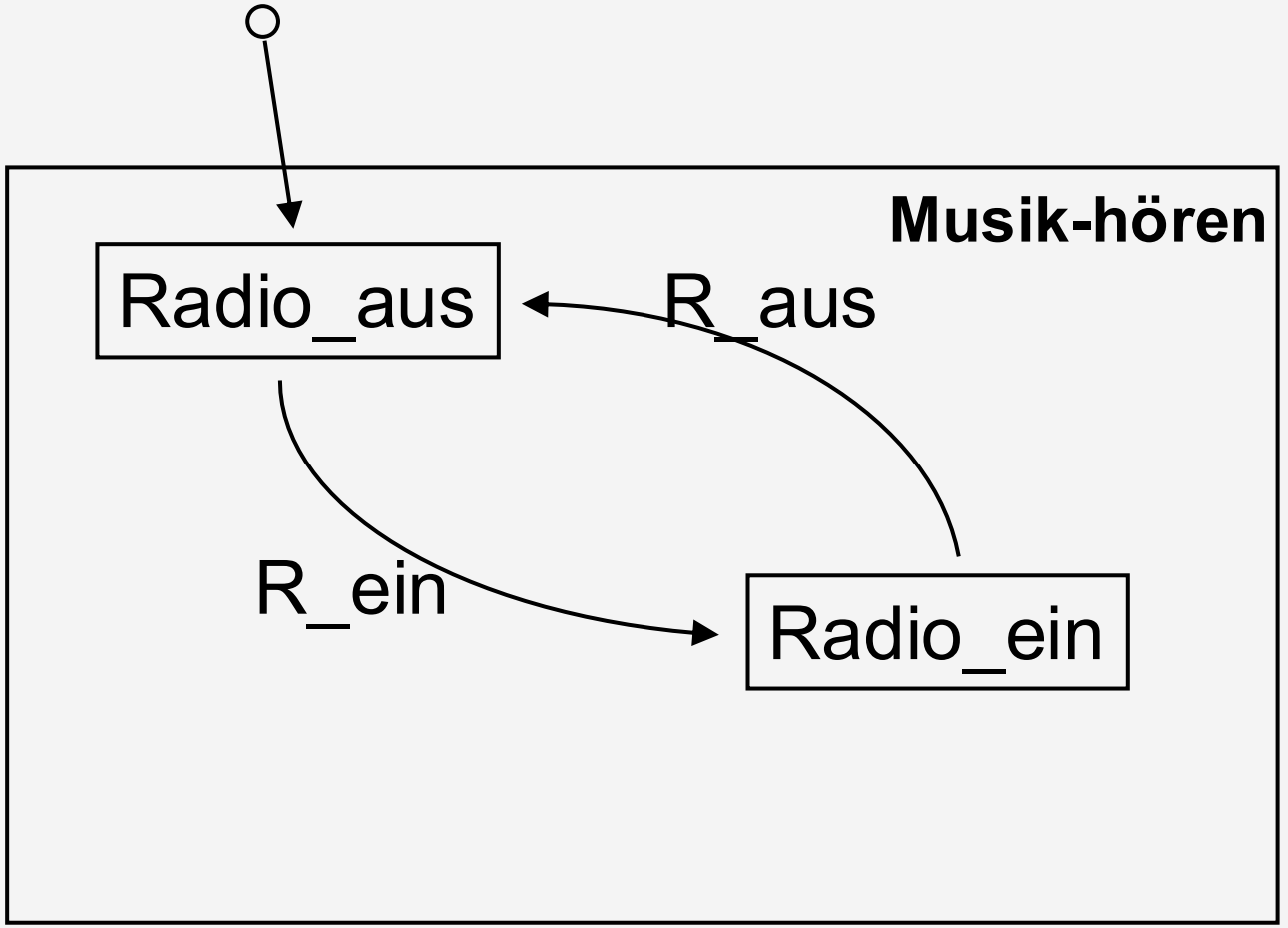
- Oberzustand aktiv genau dann wenn einer der Unterzustände aktiv
- höchstens ein Unterzustand aktiv
- schematisch dargestellt durch Schachtelung

- Vorteile
  - Reduktion: Zusammenfassen ähnlicher Zustände, insgesamt weniger Kanten
  - entspricht Entwurf mittels Verfeinerung

Beispiel Automat für Auto



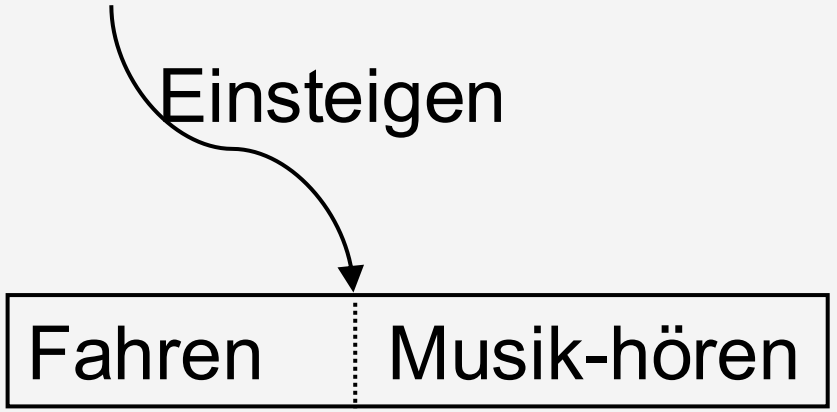
Beispiel: Musik im Auto



Und-Zustand

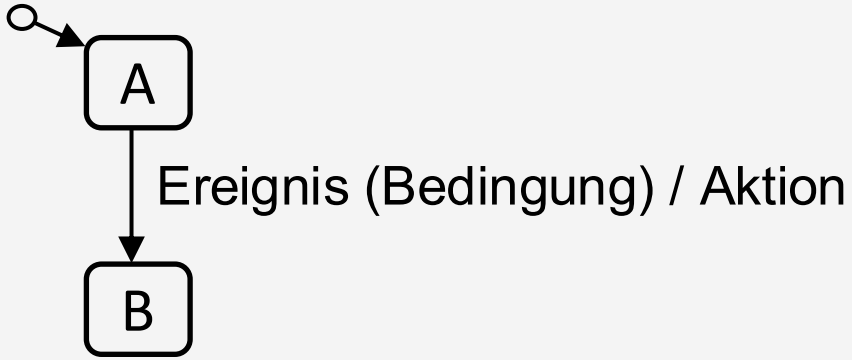
- Teilen eines Oberzustands in Teilautomaten
- Nebenläufige Ausführung der Teilautomaten
- Darstellung im Diagramm: Trennung durch gestrichelte Linien
- Bei Übergang in Und-Zustand:
  - Jeder Teilautomat wird in seinem Anfangszustand gestartet

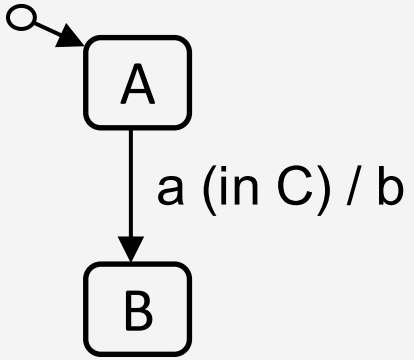
Beispiel Und-Komposition



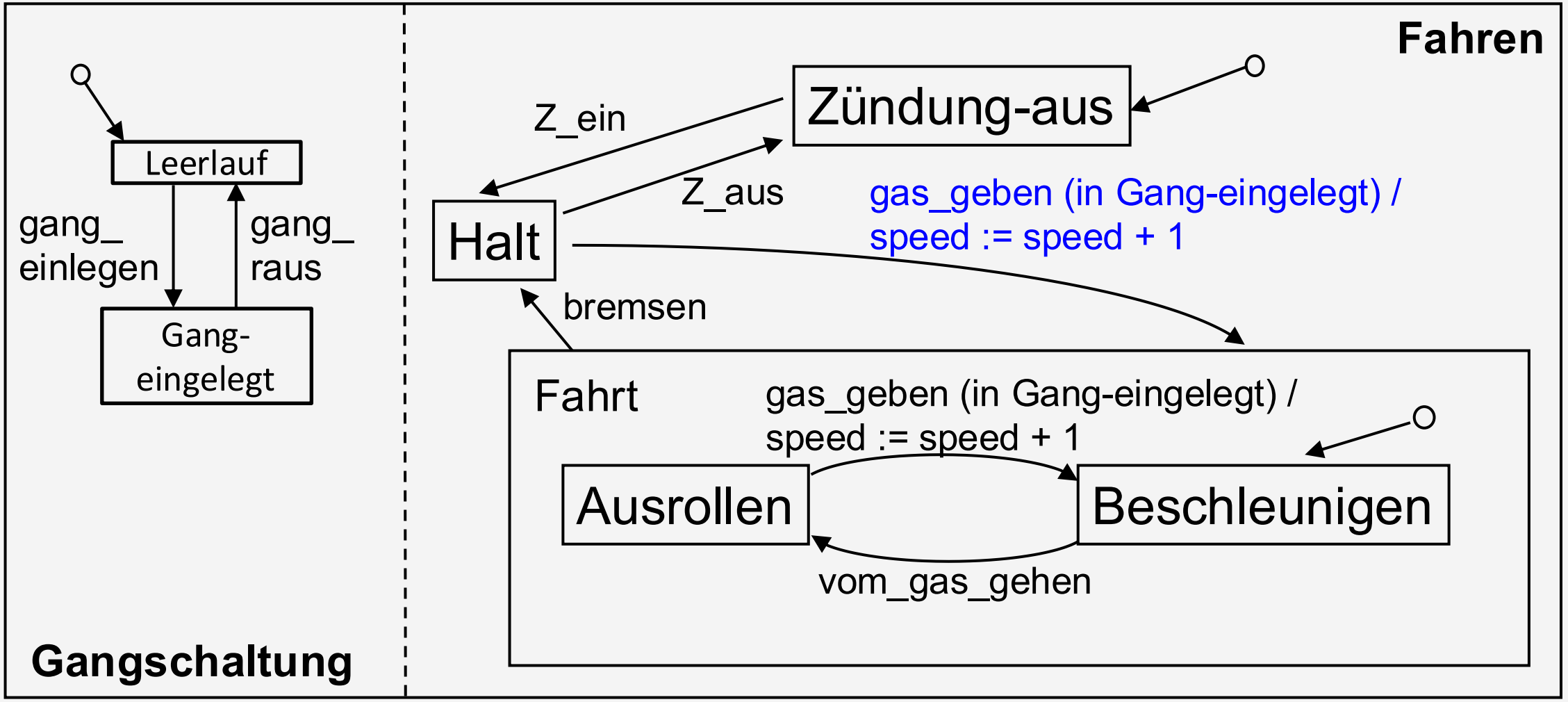
Bedingungen und Aktionen

- das Ausführen von Transitionen kann wie in endlichen Automaten an Bedingungen geknüpft werden und Aktionen auslösen
- die Bedingung kann sich auf einen parallelen/nebenläufigen Teilautomaten beziehen





Beispiel: Gangschaltung



Broadcast-Kommunikation

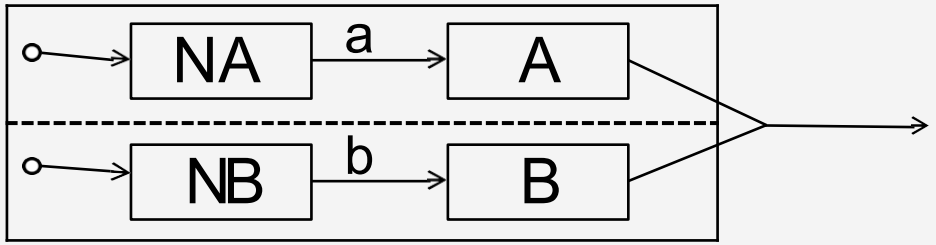
- Ereignisse in Statecharts sind global sichtbar
- das Auslösen eines Ereignisses stellt *Broadcast* an alle Teilautomaten dar
- Ereignisse können daher zur Synchronisation nebenläufiger Prozesse verwendet werden
- Eintritt eines Ereignisses löst Übergänge in allen Teilautomaten aus

Join (Zusammenführung)

- Mehrere Übergänge, ausgehend von verschiedenen Teilautomaten eines Und-Zustands, können auf gemeinsamen Nachfolgezustand führen, der nicht zum Und-Zustand gehört.
- Diese Übergänge werden gemeinsam durchgeführt.
- Voraussetzung dafür: alle Ereignisse und Bedingungen, die zu diesen Übergängen führen, müssen eingetreten sein.

Synchronisation durch Join

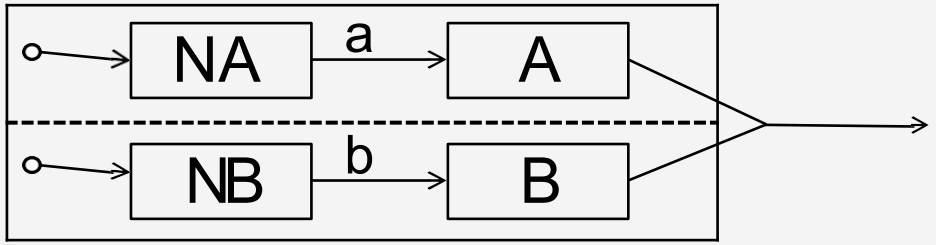
Mehrere **Übergänge aus verschiedene Teilautomaten** eines Und-Zustands können **auf gemeinsamen Nachfolgezustand** führen, der nicht zum Und-Zustand gehört.



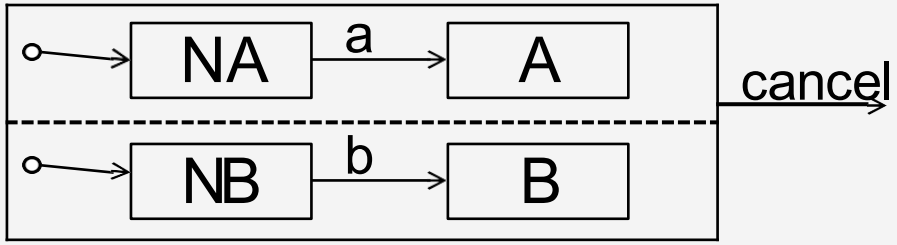
- Diese Übergänge werden **gemeinsam durchgeführt**.
- **Voraussetzung** dafür: **alle Ereignisse und Bedingungen**, die zu diesen Übergängen führen, müssen eingetreten sein.
- Im **Beispiel**: Übergang aus dem Und-Zustand hinaus **erst nachdem beide Ereignisse eintreten** (a und b)

Synchronisation durch Join

Mehrere **Übergänge aus verschiedene Teilautomaten** eines Und-Zustands können **auf gemeinsamen Nachfolgezustand** führen, der nicht zum Und-Zustand gehört.

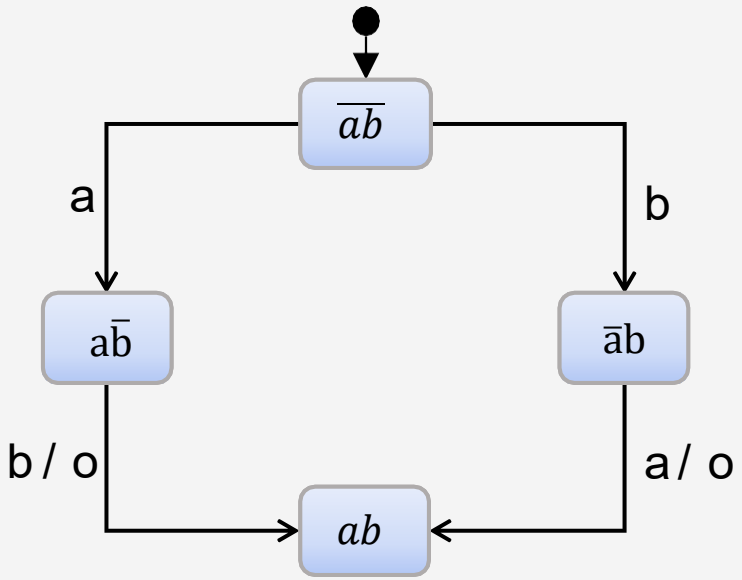


Im Gegensatz dazu erfolgt hier der Übergang aus allen Unterzuständen, möglicherweise bevor *a* oder *b* auftreten:

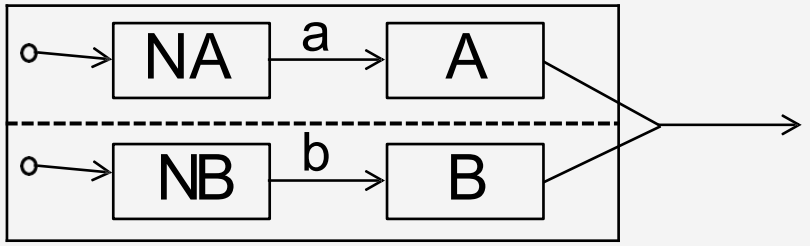


Nebenläufigkeit ohne Zustandsexplosion

Endlicher Automat



Statechart

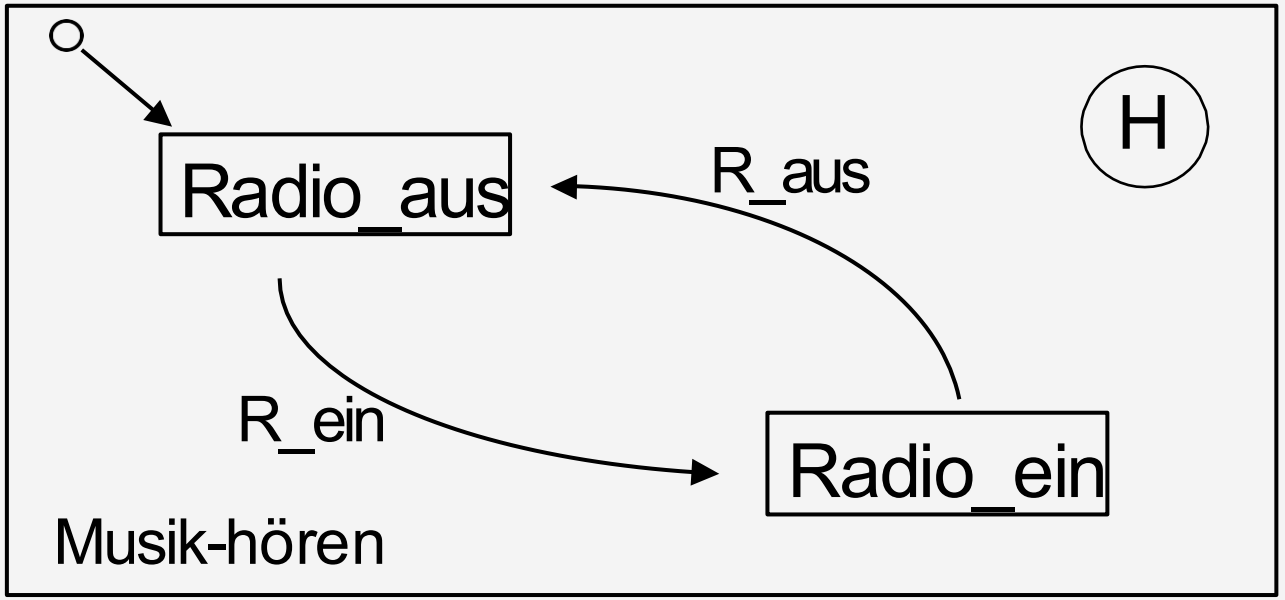


Erweiterbar: ähnliche Teilautomaten

History (Gedächtnis)

- Problem: Bei Eintritt in Oberzustand soll vorangegangener Zustand wieder eingenommen werden.
- Gelöst durch History (H)
  - erinnert letzten Zustand auf oberster Ebene
- Rekursive History (H^*)
  - auch rekursiv für alle internen Zustände

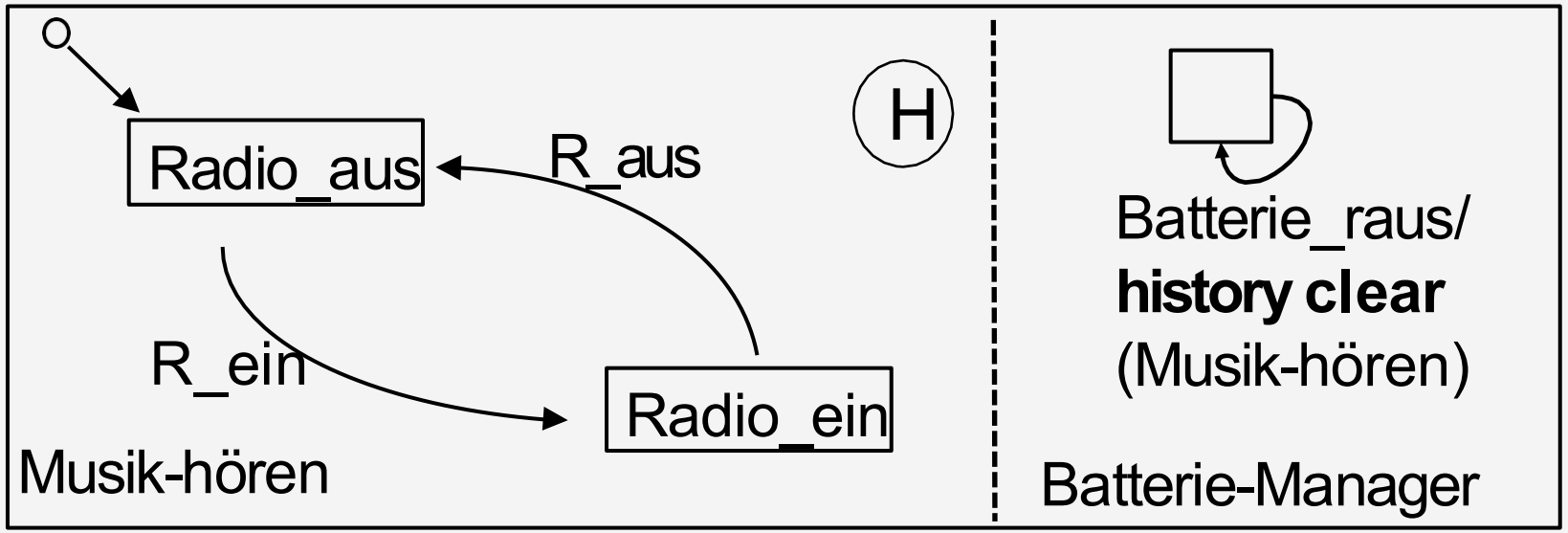
Beispiel: Musik im Auto mit Historie



Beim **ersten Betreten** des Oberzustands ist das **Radio aus**, **danach** wird immer **der zuletzt besuchte Zustand** betreten.

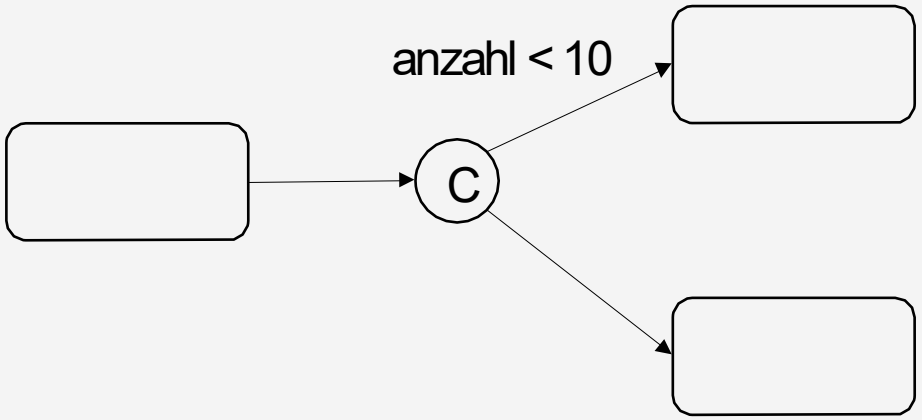
Zurücksetzen der History

- Beispiel: Herausnahme der Autobatterie löscht Speicher des Radiozustands



Konditionalkonnektor

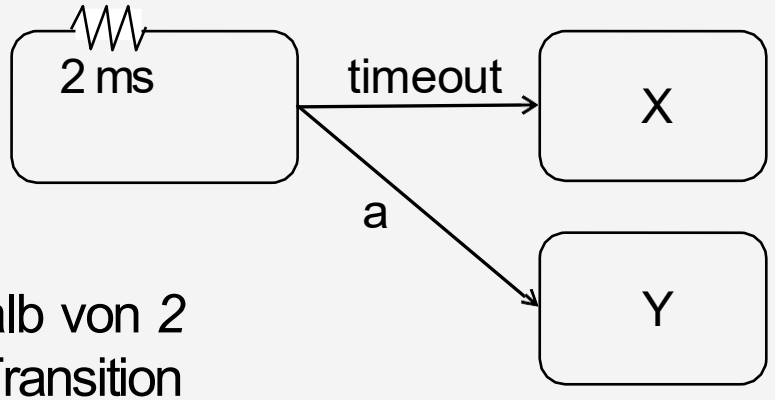
- Übersichtliche Darstellung einer Fallunterscheidung
- Kreis mit „C“ gekennzeichnet
- davon ausgehend mehrere Transitionen (Verzweigung)
- mind. eine Default-Transition ohne Trigger/Bedingung



Timer

- Zeit für eingebettete Systeme sehr wichtig
- Timer mit Timeout-Events:

timeout (*event*, *number*)



Wenn *a* nicht innerhalb von *2 ms* eintritt, wird die Transition zu *X* genommen.

Entry-, Exit-, Throughout Actions

- Spezifizieren Aktionen, die beim Betreten, Verlassen oder während des Aufenthalts in einem Zustand ausgeführt werden

- **entry** event1, var1 := 0 **exit** event2 **throughout** event3

Je nach Statechart-Dialekt wird statt **throughout** auch **during** verwendet oder eine Zeitangabe wie z.B. **every 5 sec**

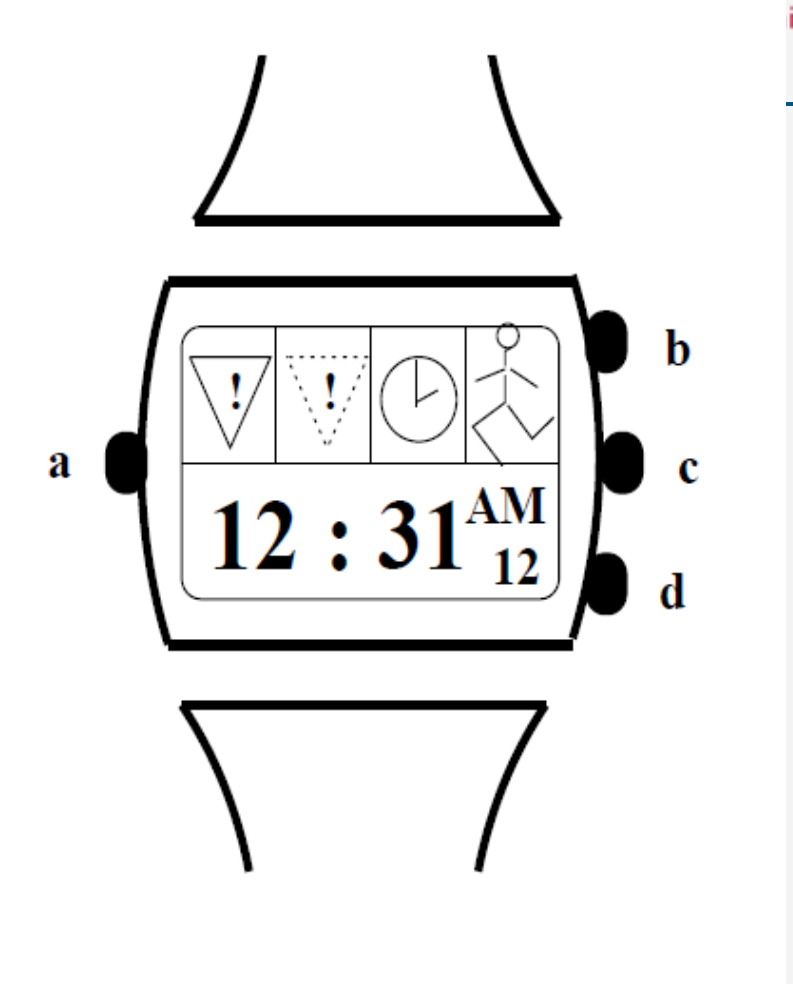
Charakterisierung von Statecharts

Statecharts =

Zustandsautomat
+ Hierarchie
+ Orthogonalität (Unabhängigkeit)
+ Broadcast-Kommunikation
+ History

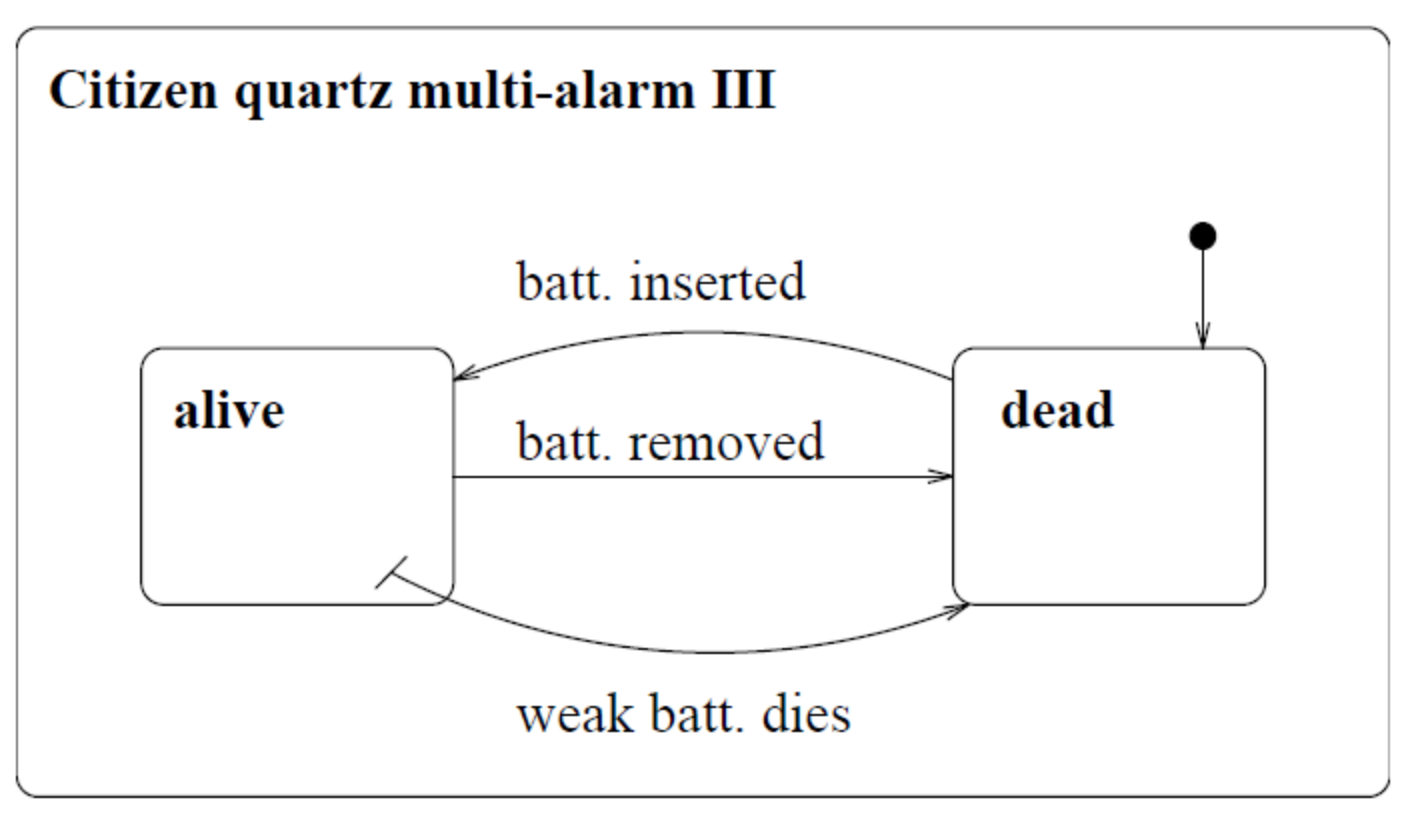
Beispiel (von Harel): Digitale Uhr

- Funktionen:
  - Zeitangabe
  - Stundenbeep, Beepertest
  - Zwei unabhängige Alarme
  - Stoppuhr
  - Licht, Stromschwächenanzeige

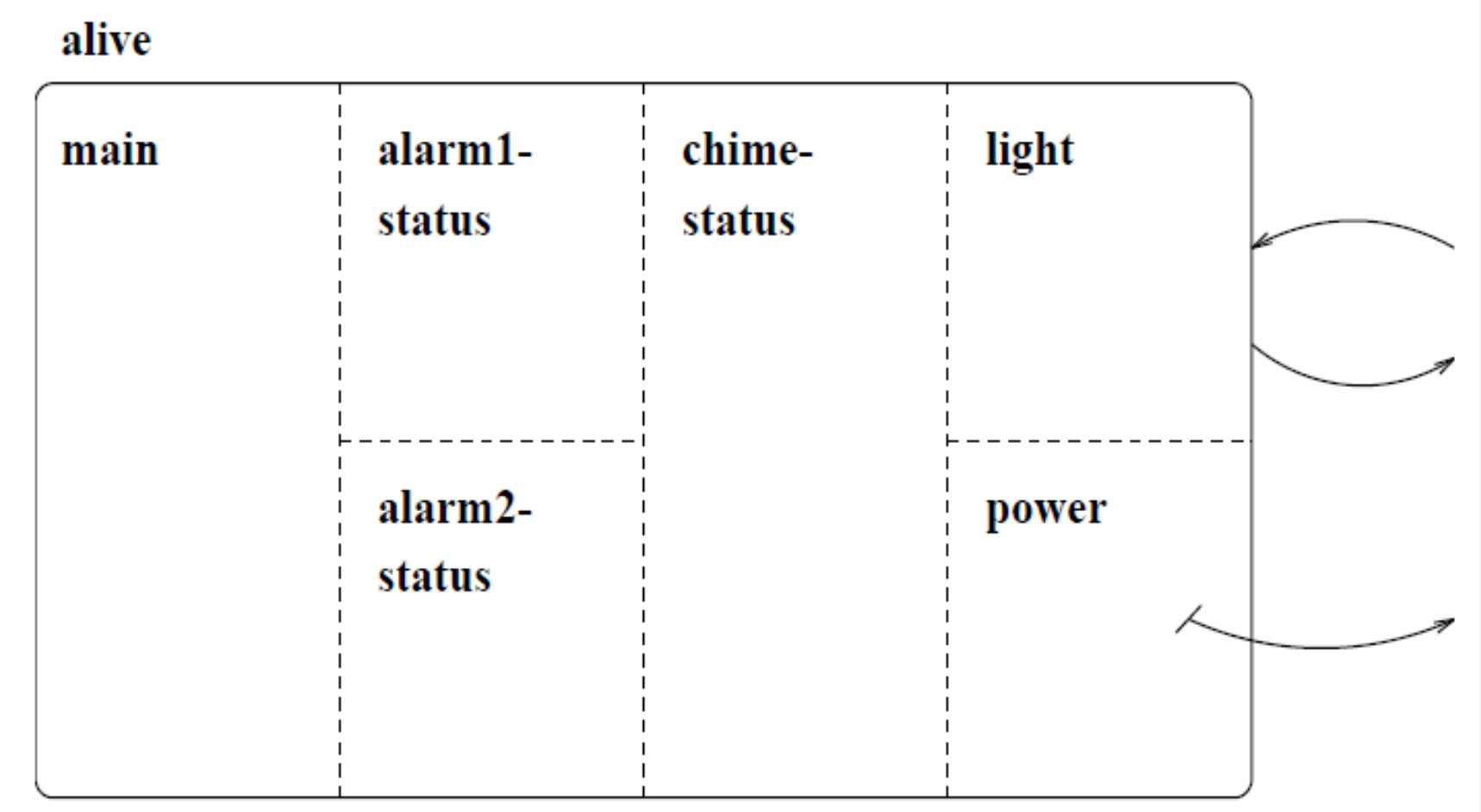


Zeichnen Sie ein Statechart, das das Verhalten dieser Uhr modelliert!

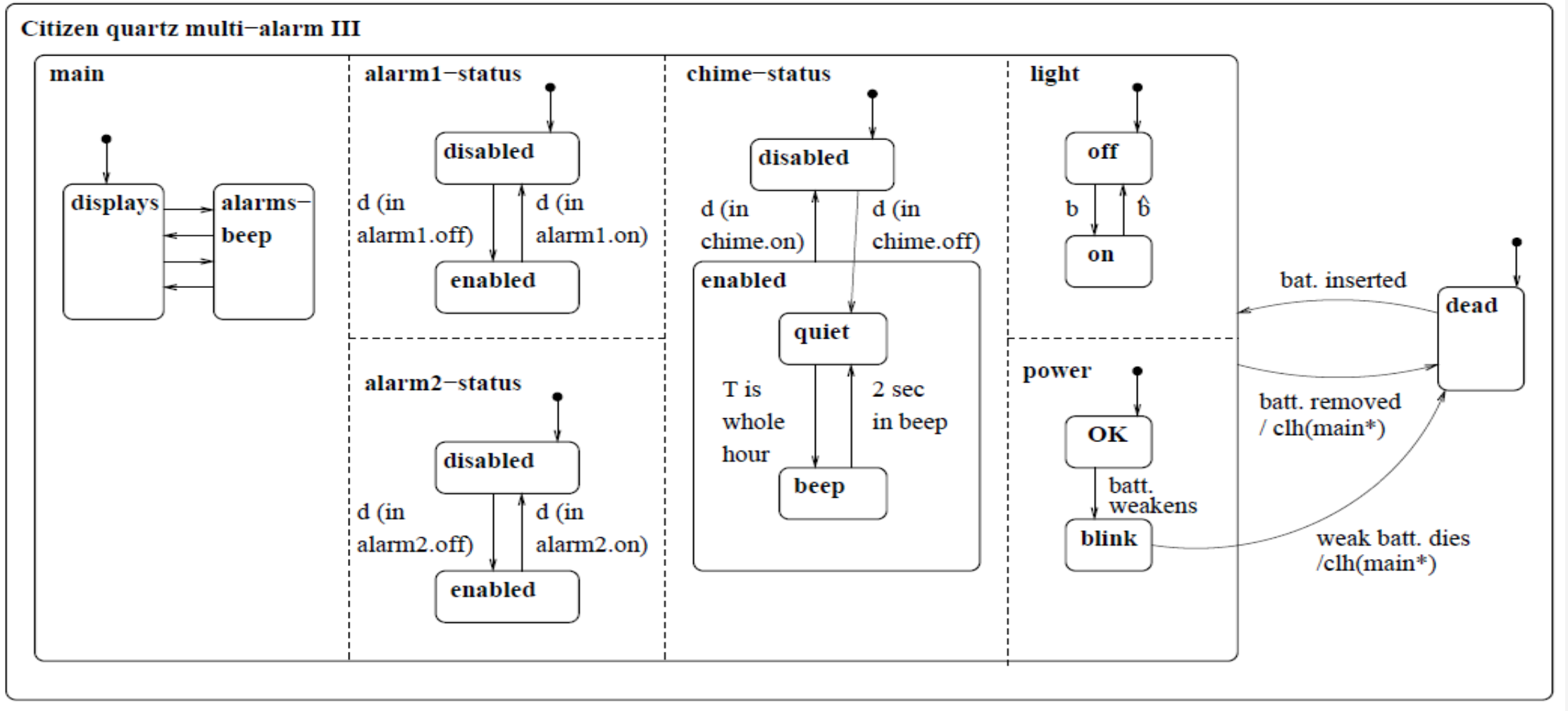
Beispiel (von Harel): Top-Level



Beispiel (von Harel): Erste Verfeinerung

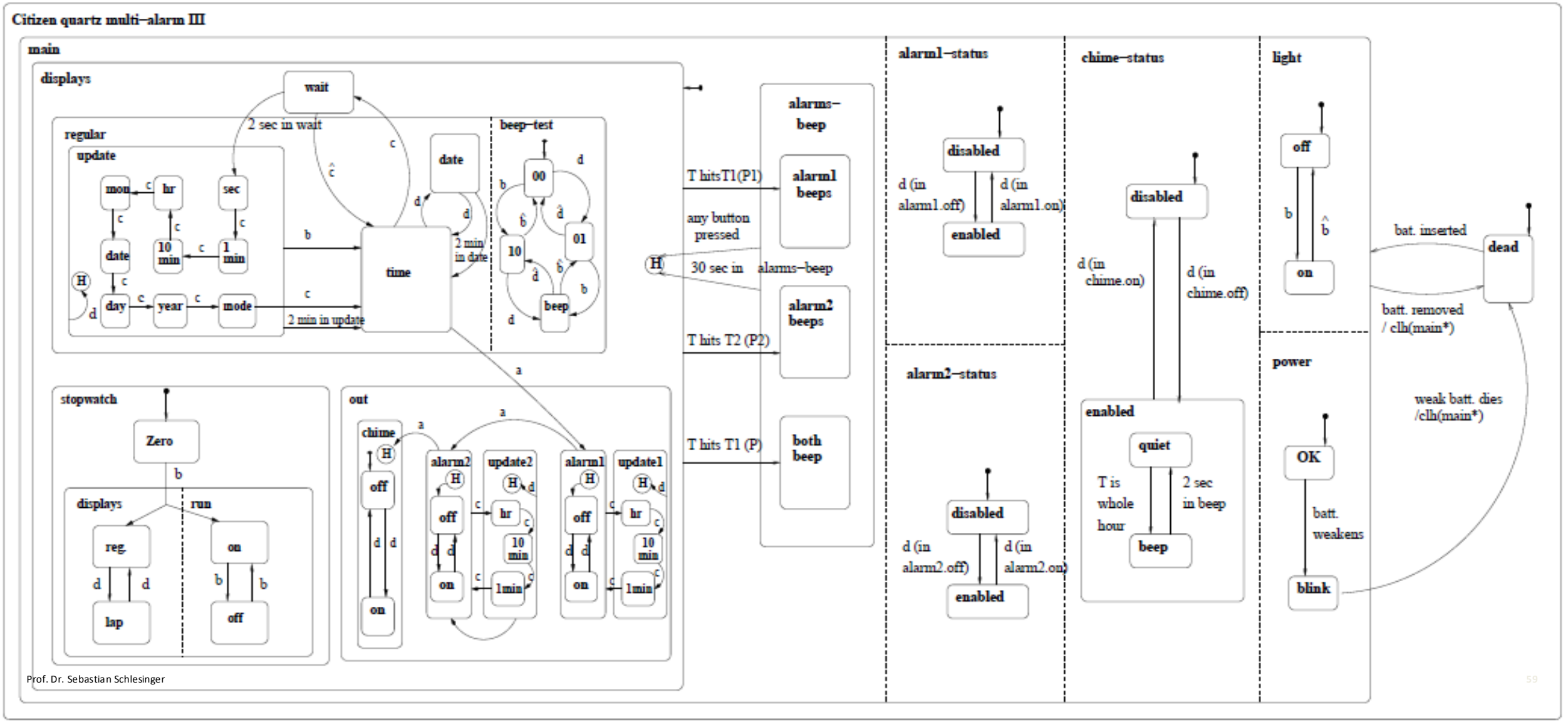


Beispiel (von Harel): Zweite Verfeinerung



Beispiel (von Harel): Gesamtspezifikation

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law



Aktionen und Datenvariablen

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Es existieren verschiedene Statechart „Dialekte“ (z.B. UML, Stateflow)
- wichtige Erweiterung: Aktionen und Datenvariablen
- z.B. Funktionsaufrufe oder Zuweisungen an Transitionen oder als „entry“ und „exit“ in Zuständen (entry-Aktionen werden beim Betreten des Zustands ausgeführt, exit-Aktionen beim Verlassen)

Syntax und Semantik

- Es existieren verschiedene Statechart „Dialekte“
- Syntax ist bei allen Statechart-Arten (fast) gleich
  - Zustände = Kästen oder Kreise
  - Zustandsübergänge = Pfeile
  - …
- Semantik abhängig von gewählter Statechart-Art

Formale Semantik

Formale Semantik:
weist syntaktischen Elementen formale Bedeutung zu

In mathematischer Logik:

Aussagenlogik: Formel A v B

Semantik: Wahrheitstafel

A v B	A = 0	A = 1
B = 0	0	1
B = 1	1	1

Für Programmiersprachen:

Programmiersprachen: Programme

```
x:=1;
while x<3 do
    x:=x+1
```

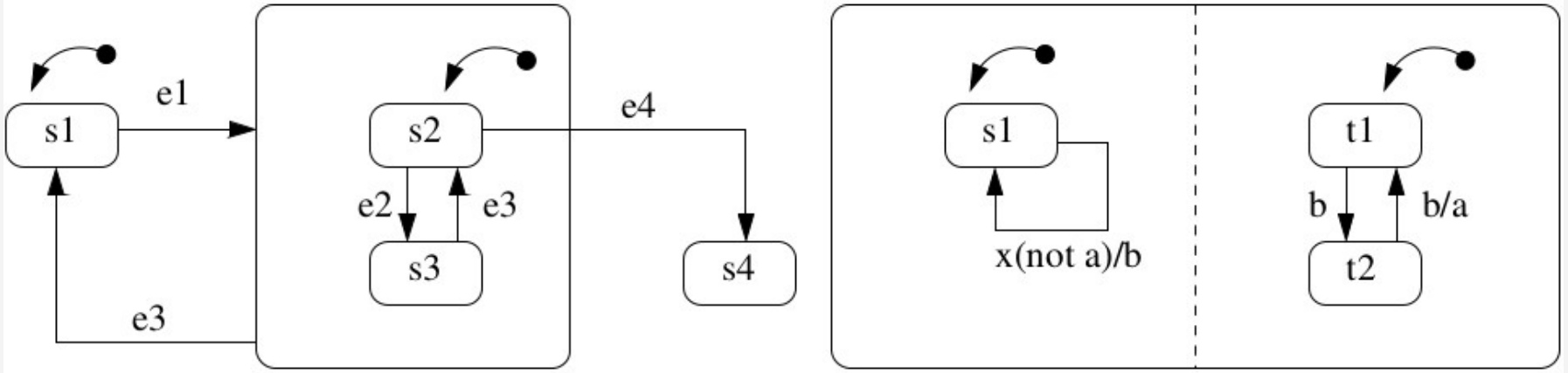
Semantik:
- denotationell: Ergebnis der Berechnung (sinnvoll bei funktionalen Sprachen)
- operational: Zustandsübergangsfolge (sinnvoll bei imperativen Sprachen))

Semantik für Beispielprogramm:
- Ergebnis der Berechnung: (x = 3)

- Zustandsübergangsfolge:
  (Z. 1, x = ?) → (Z. 2, x = 1) → (Z. 3, x = 1) → (Z. 2, x = 2) → (Z. 3, x = 2) → (Z. 2, x = 3) → (Z. 4, x = 3) → …

Semantik: mehrere Möglichkeiten

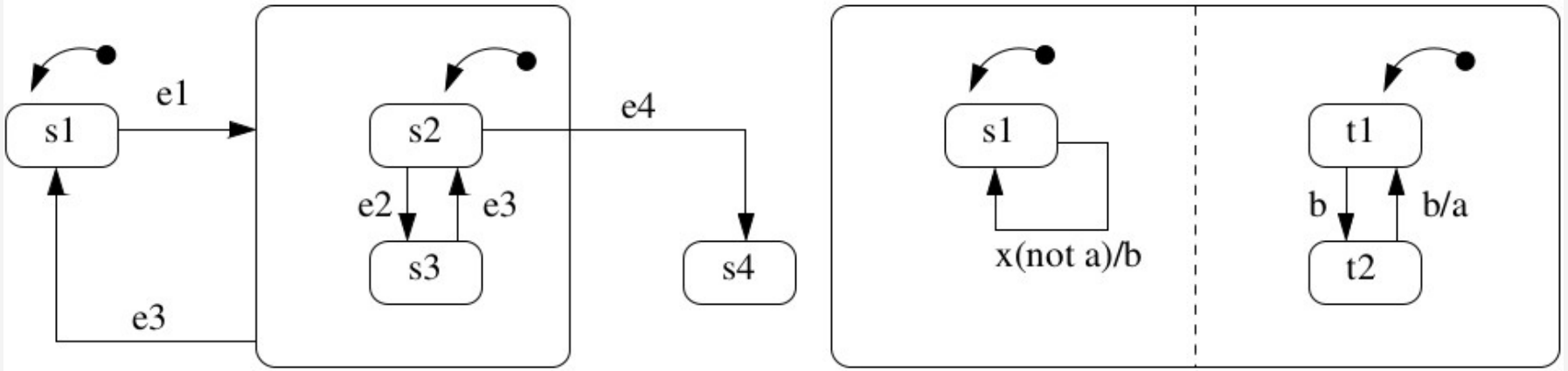
- In den folgenden Beispielen sind mehrere Übergangsfolgen mit unterschiedlichem Ergebnis möglich



(a) (b)

- Was passiert in (a), wenn das System zuerst die Ereignisse e1 und e2 empfängt und dann e3 eintritt?
- Was passiert in (b), wenn das System die Ereignisse x, x empfängt?

Semantik: mehrere Möglichkeiten



(a) (b)

a) Das System ist im Zustand s3 und kann nun mit e3 entweder nach s1 oder nach s2 gehen.
=> Child-first oder Parent-First execution

b) Beim ersten x wird b ausgegeben. Dann kann der Übergang von t1 nach t2 genommen werden und dann gibt es einen Konflikt. Bei synchroner Semantik kann beim zweiten x die Transition
t2->t1 gleichzeitig passieren und damit die Transition s1->s1 ungültig machen (gleichzeitig not a und a)
=> Sichtbarkeit gesendeter Ereignisse muss definiert werden

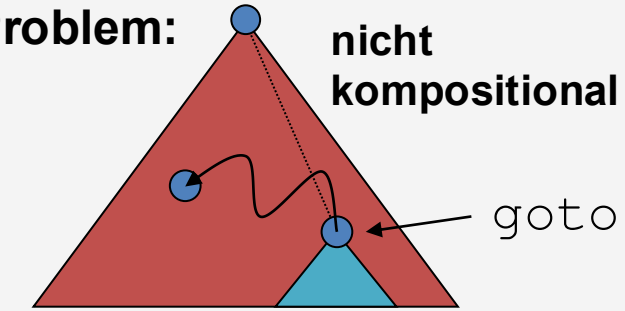
Exkurs: Kompositionalität der Semantik

In der mathematischen Logik:
Wahrheitswert einer Formel:
- aus Wahrheitswerten der Teilformeln

F = **A** v **B**

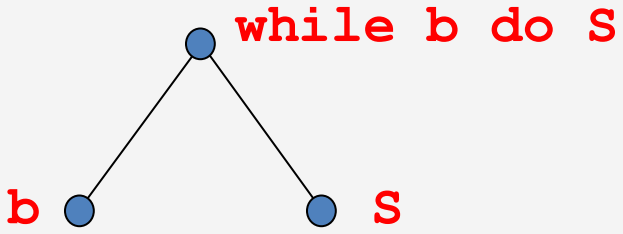
F ist wahr gdw.
A ist wahr oder **B** ist wahr

Problem:



Bei Programmiersprachen:
Semantik eines Programms:
- aus Semantik seiner direkten Teile
- rekursiv: auch aus der eigenen Semantik

Programm = abstrakter Syntaxbaum (AST)



`while b do S` =
if `b` then (`S`; `while b do S`)
else ;

Kompositionalität der Semantik

Warum ist das wichtig?

Kompositionalität der Semantik

- Kompositionalität: Bedeutung des Ganzen = Bedeutung der Teile + Kombinationsregel
- Ohne Kompositionalität: ganzes System analysieren
- Mit Kompositionalität: Teile analysieren, Kontrakte übertragen sich auf System, d.h. Verifikation der Teile überträgt sich auf Gesamtsystem
- Was ist kompositional?
  - Programmiersprachen: denotationelle Semantik: ja, operationale Semantik: nicht vollständig (Verhalten hängt vom Kontext wie Speicherzustand, Scheduler, Umbegung ab)

Was ist mit FSMs und State Charts?

Kompositionalität von FSMs und State Charts

- Sind FSMs kompositional?
  - Ja, Komposition zweier FSMs $M_1 = (Q_1, \Sigma_1, \delta_1, q_{0,1}), M_2 = (Q_2, \Sigma_2, \delta_2, q_{0,2})$ ist $M_1 || M_2 = (Q_1 \times Q_2, \Sigma, \delta, (q_{0,1}, q_{0,2})), \delta((q_1, q_2)) = (\delta_1(q_1, a), \delta_2(q_2, a))$

- Sind Statecharts kompositional?
  - Nein
  - Globale Event-Broadcasts, Hierarchie, Transitionen über Ebenen etc. brechen es
  - Ebenso synchrone Semantik (ein Event beeinflusst alles)
  - Kompositionalität wird besser bei asynchroner Semantik

Semantik von Statecharts

- Es existieren viele verschiedene Semantiken
  (z.B. UML, Stateflow, Statemate, Safe State Machines)
- Bestandteile der Semantik
  - Was ist ein Zustand?
  - Wie finden Zustandsübergänge (Transitionen) statt?
  - Ereignisverarbeitung
  - Auflösung von Konflikten
- Allgemein:
  - Menge aktiver Zustände (+ Variablenbelegungen) = „semantischer“ Zustand

Synchrone Abarbeitung

- Mögliche Semantik für Statecharts
- In jedem Zeittakt:
  - einer, mehrere oder alle Teilautomaten können einen Übergang durchführen
- Kombination aller möglichen Übergänge
  - ergibt mögliche Übergänge des sequentiellen Prozesses
- Synchroner Ablauf

Asynchrone Abarbeitung

- Weitere mögliche Semantik für Statecharts
- Kein einheitlicher Takt
  - Auslösen und Abrufen von Events passiert nicht gleichzeitig
    - Ausgelöste Events landen im Eventspeicher
    - Können „irgendwann“ abgerufen werden
  - Teilautomaten führen Übergänge unabhängig voneinander durch
- Asynchroner Ablauf

Synchrone vs. Asynchrone Abarbeitung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Diskutieren Sie Vor- und Nachteile synchroner vs. asynchroner Abarbeitung!

Synchrone vs. Asynchrone Abarbeitung

Synchron

- ✓ besser vorhersagbar
- ✓ statisch analysierbar
- ▪ in der Regel weniger performant als asynchrone Abarbeitung

- Synchron für feste Zeitintervalle
  - Alle 10ms soll der Messwert übermittelt werden
- Asynchron für Ereignissteuerung
  - Wenn die Temperatur über einen bestimmten Wert steigt, soll die Kühlung angeschaltet werden

Tool: Itemis CREATE

Alter Name: *Yakindu*

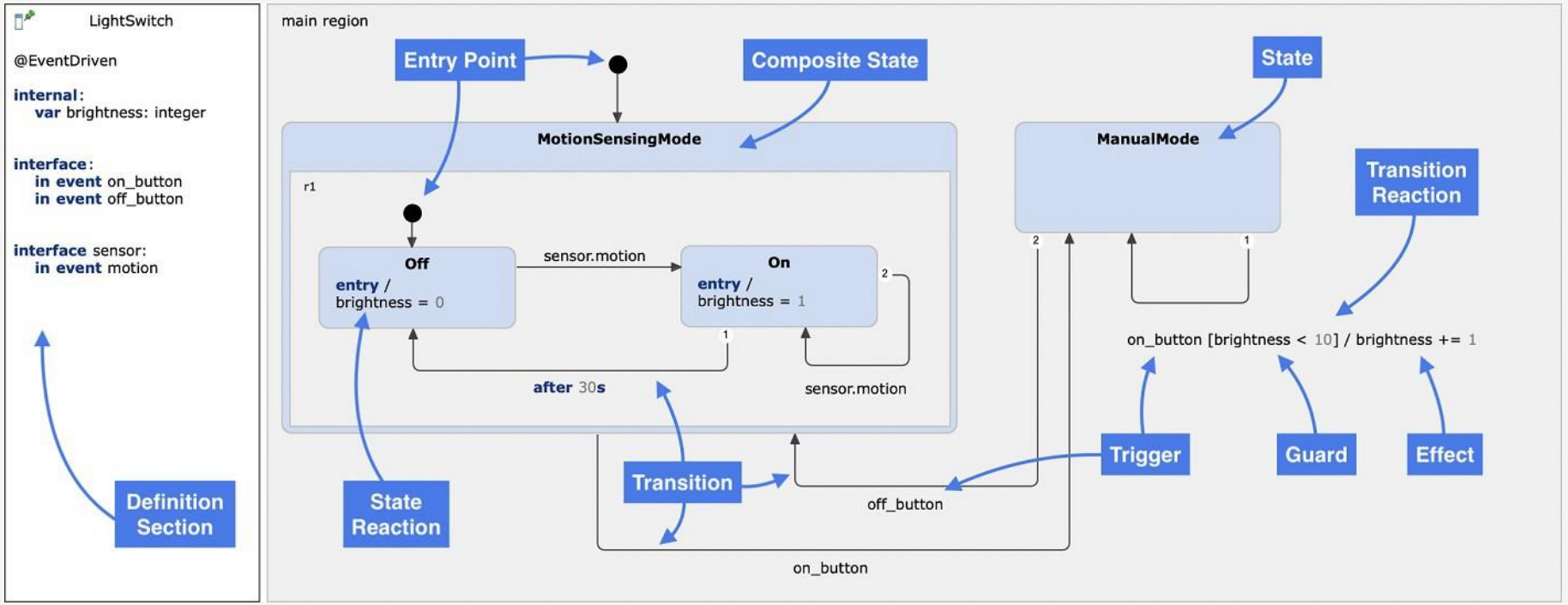
- Modellierung, Simulation, Testen und Code generieren
  - animierte Simulation
  - Unit Tests für Statecharts
  - Code Generatoren für C, C++, Java, Python
  - basiert auf Eclipse

- Link: https://www.itemis.com
- Academic Use ist frei → Lizenzformular ausfüllen

CREATE: Besonderheiten

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Variablen und Events werden im Interface deklariert
- Prioritäten an Transitionen und nebenläufigen Regionen

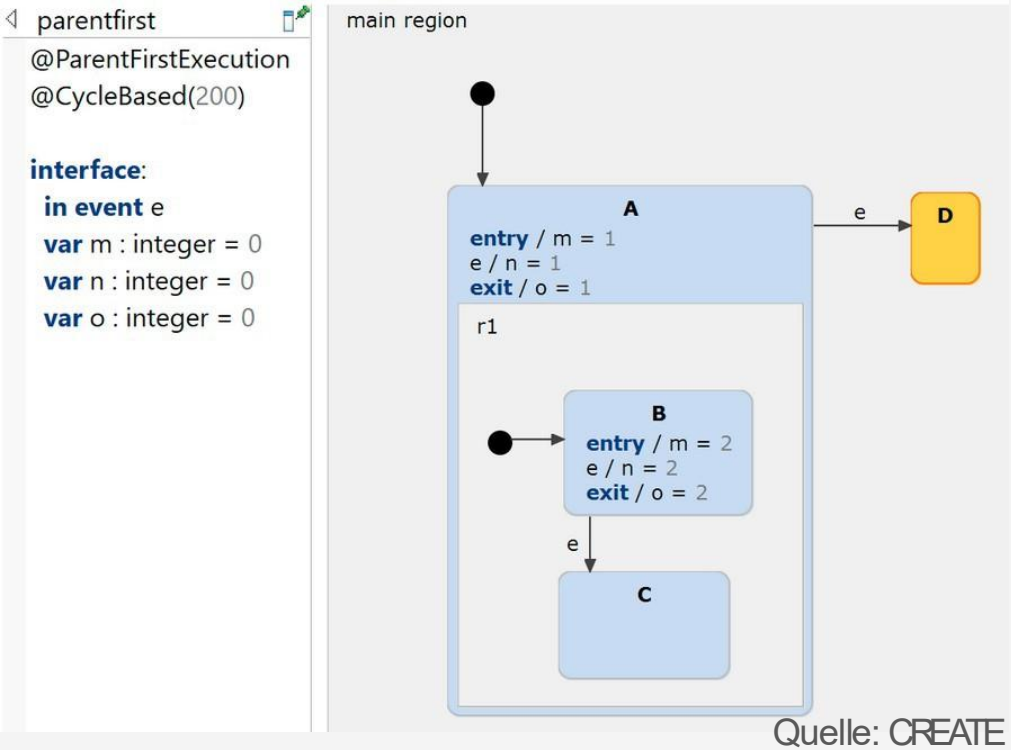


- Zusätzliche Syntaxelemente, zB.

Final State | Exit Node | Choice | Synchronization

CREATE: Semantik

- Oberzustände können *parent-first* oder *child-first* ausgeführt werden
- Unterscheidung zwischen event-driven und cycle-based execution



Quelle: CREATE Documentation

CREATE: run-to-completion steps

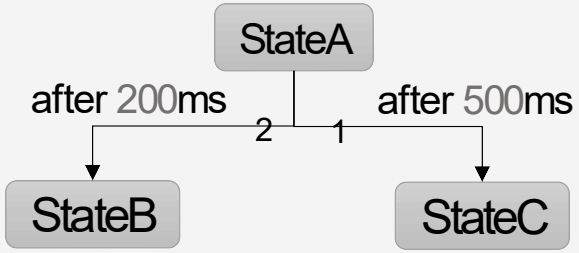
- Ausführung basiert auf run-to-completion (RTC) steps
  - Events triggern Transitionen
  - Abhängig von gewählter Ausführung
    - Cycle-based: Eingangs- und Zeitereignisse werden gesammelt und zu festen Zeitpunkten verarbeitet
    - Event-driven: jedes Ereignis wird sofort in einem eigenen RTC verarbeitet

CREATE: cycle-based Execution

- Zeitgesteuert (cycle-based): periodische Ausführung

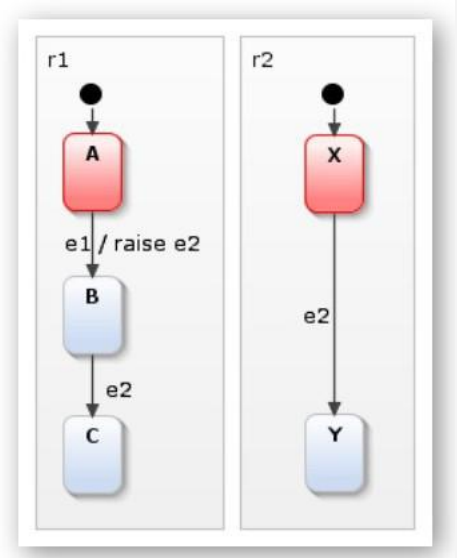
  (Intervall kann definiert werden)

  - Bedingungen werden nur alle CYCLE_TIME ms geprüft



Bei CYCLE_TIME = 1000ms sind beide Transitionen möglich (Priorität entscheidet), bei 200ms nur die zu StateB.

- Events werden gesammelt und Transitionen gemäß Ausführungsreihenfolge ausgeführt
  - ermöglicht Bedingungen wie [Event A & Event B]
  - Events sind nur in noch nicht ausgeführten Orthogonal States sichtbar! (down-stream)

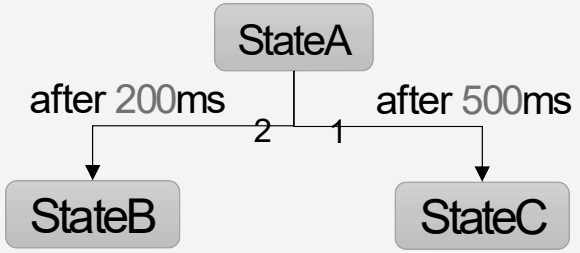


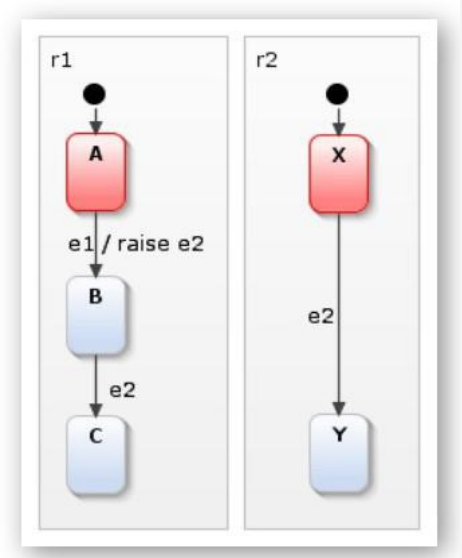
Quelle: CREATE Documentation

CREATE: event-driven Execution

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

- Ereignisgesteuert (event-driven): events werden sofort verarbeitet
  - jedes Event triggert sofort (nach der Ausführung des aktuellen Steps) einen run-to-completion (RTC) step
  - Bedingung [Event A & Event B] nicht möglich, da jedes Event seinen eigenen RTC hat

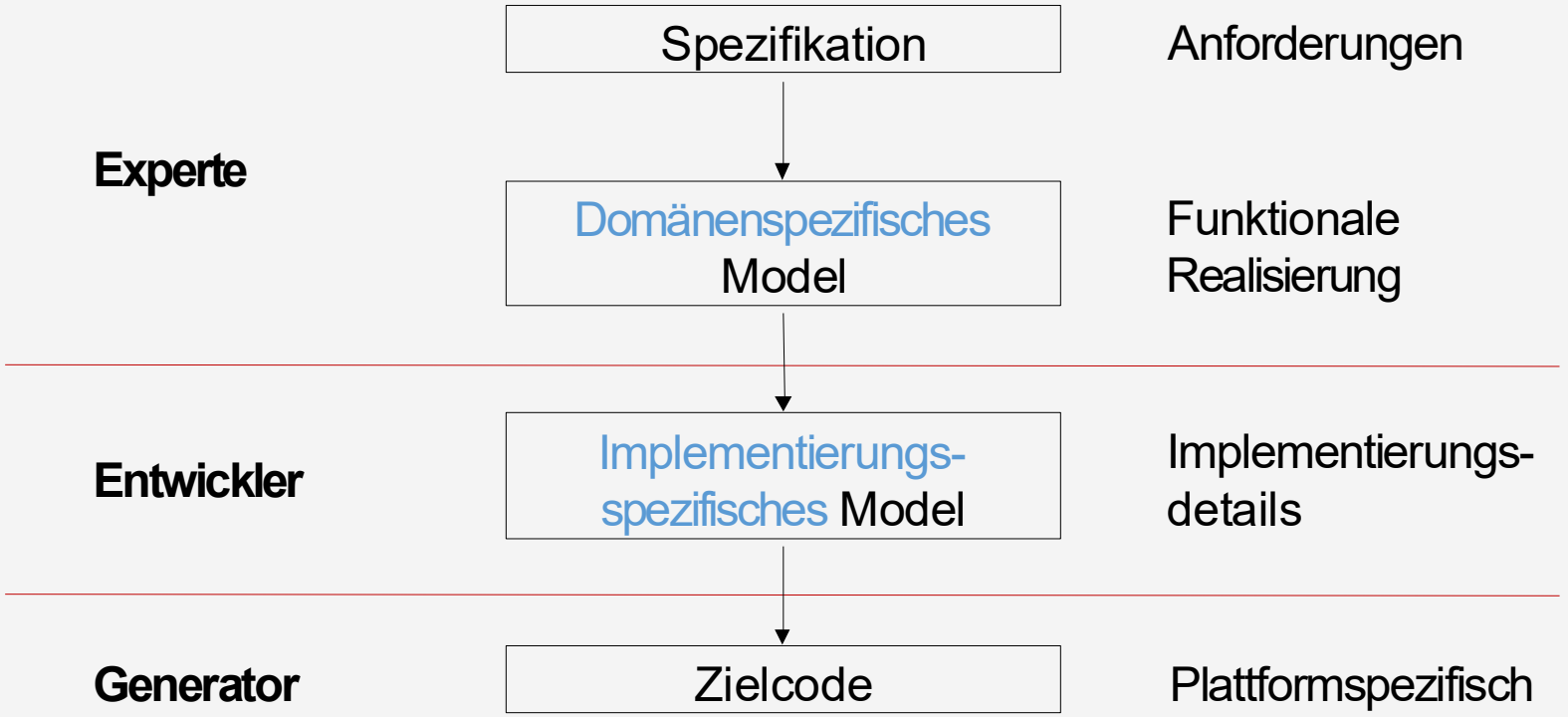




Quelle: CREATE Documentation

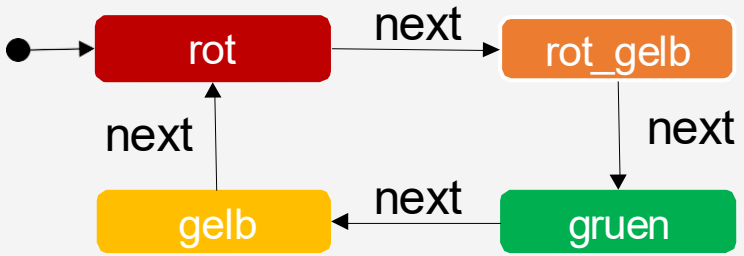
Model-Driven Development (MDD)

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law



MDD mit Statecharts

- C Code generieren für Statecharts




```
typedef int signaltype;
signaltype next;

typedef int statetype;
statetype rot, rot_gelb, gruen, gelb;

int main ( ){
  int done;
  do {
    get_inputs();
    done  =  tick();
    put_outputs();
  } while (!done);
  return 0;
}
```

```
int tick(){
  switch (state){
    case rot:
      if (next)
        state = rot_gelb;
      break;
    case rot_gelb:
      if (next)
        state = gruen;
      break;

    . . .

    case gelb:
      if (next)
        state = rot_gelb;
      break;
  }
  return 0;
}
```

MDD mit CREATE

- CREATE bietet Code Generatoren für C, C++, Java und Python
- Der generierte Code
  - stellt Interfaces zur Verfügung für den Zugriff auf Variablen und Events.
  - initialisiert das Statechart.
  - implementiert die Ausführungssemantik innerhalb eines run-to-completion (RTC) step.

Zusammenfassung

- Endliche Automaten
  - Modellierung von Hard- und Software
  - eindeutig, simulierbar und verifizierbar
  - keine Hierarchie, Verfeinerung, Modularisierung, Zustandsexplosion bei Nebenläufigkeit
- Nebenläufigkeit
  - vermeidet Überspezifikation, erhöht das Optimierungspotential
  - ermöglicht einen modularen/verteilten Entwurf
  - Problem der kombinatorischen Zustandsexplosion
- Statecharts
  - erweitern endliche Automaten um Nebenläufigkeit, Hierarchie, Broadcast und History
  - intuitiv gut verständlich, graphische Notation
  - sehr gut zur Modellierung reaktiver und eingebetteter Systeme geeignet
  - Zustandsexplosion wird graphisch vermieden, tritt aber (immer noch) bei der Verifikation auf
  - Semantik nicht immer eindeutig

Key Takeaway

FSM:
+ einfach
+ formal
– keine Nebenläufigkeit
– Zustandsexplosion

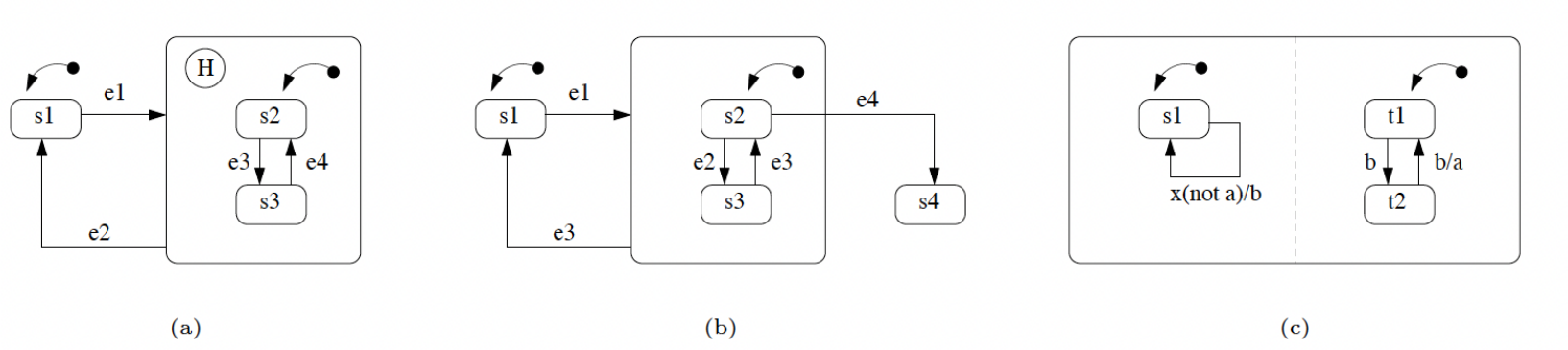
Statecharts:
+ Hierarchie
+ Nebenläufigkeit
– Semantik komplex
– Verifikation schwierig

Next Step: Timed Automata

- Timed Automata modellieren Zeit in ihrer Semantik direct
- Auch in State Charts ist Zeit (per Erweiterung) modellierbar, aber nur indirekt (nach Zeitablauf wird Signal emittiert), nicht als Teil ihrer Semantik
- Statecharts lösen das Strukturproblem – Timed Automata das Analyseproblem

Übung / Reflexion

1. Welche Art von Systemen können mit Statecharts gut, welche weniger gut modelliert werden? Diskutieren Sie Beispiele!
2. Welche Probleme muss die Semantik von Statecharts lösen? Wann kann es Mehrdeutigkeiten geben?
3. Was ist der Unterschied zwischen einer synchronen und einer asynchronen Semantik?



4. Betrachten Sie die o.g. StateCharts.
   1. Das System (a) empfängt die Ereignisse e1, e3, e2, e1. In welchem Zustand ist das System nach jedem Schritt?
   2. Das System (b) empfängt die Ereignisse e1, e2. In welchem Zustand ist das System nach jedem Schritt? Was passiert, wenn nun e3 eintritt? Warum? Was muss danach passieren, um s4 zu erreichen?
   3. Das System (c) empfängt die Ereignisse x, x. Welchen Konflikt gibt es? Wie laässt er sich auflösen oder verhindern?

Literatur

- David Harel: Statecharts: A Visual Formalism for Complex Systems. Science of Computer Programming, Vol. 8, No. 3, S. 231-274, 1987.
- David Harel: On Visual Formalisms. Communications of the ACM (CACM), Vol. 31, No. 5, May 1988.